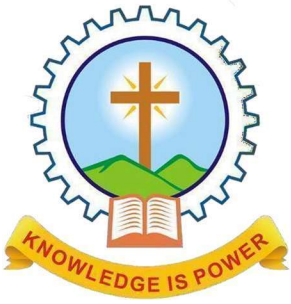
MAR ATHANASIUS COLLEGE OF ENGINEERING

KOTHAMANGALAM

(Government Aided Autonomous Institution)



Department of Computer Applications

Mini Project Report

CLAUSEIQ: MACHINE LEARNING-BASED CONTRACT CLAUSE CLASSIFICATION AND INTELLIGENT SEARCH SYSTEM

Done by

MOHAMMED FARHAN K

Reg No: MAC25MCA-2042

Under the guidance of

Dr. Sonia Abraham

2025–2027

MAR ATHANASIUS COLLEGE OF ENGINEERING

KOTHAMANGALAM

(Government Aided Autonomous Institution)

CERTIFICATE

ClauseIQ: Machine Learning-Based Contract Clause Classification and Intelligent Search System

Certified that this is the bonafide record of project work done by

MOHAMMED FARHAN K

Reg No: MAC25MCA-2042

during the third semester, in partial fulfilment of requirements for award of the degree,

Master of Computer Applications

of

APJ Abdul Kalam Technological University, Thiruvananthapuram

Dr. Sonia Abraham
Faculty Guide

Dr. Sonia Abraham
Head of the Department

Dr. Manu John
Project Coordinator

Internal Examiners

Acknowledgement

First and foremost, I express my gratitude to God Almighty for showering blessings and granting the strength, perseverance, and wisdom to complete this mini-project work successfully.

I express my profound gratitude and heartfelt thanks to Dr. Sonia Abraham, Head of the Department of Computer Applications, Mar Athanasius College of Engineering, Kothamangalam, for providing excellent academic facilities, lab resources, and continuous encouragement throughout this endeavour.

I extend my deep gratitude and sincere appreciation to my project guide, Dr. Sonia Abraham, for invaluable guidance, constructive suggestions, insightful feedback, and patient support rendered during all stages of this project development.

I express my sincere gratitude to the Project Coordinators, Dr. Manu John and Prof. Minu Maria Joseph, as well as the faculty members and technical staff of the Department of Computer Applications, for their kind cooperation, valuable guidance, and assistance.

Finally, I express my deepest appreciation to my parents, family members, and friends for their continuous moral support, understanding, and encouragement throughout the completion of this project.

Mohammed Farhan K

Abstract

Commercial contracts govern mission-critical enterprise relationships, but manual review, classification, and retrieval of operative clauses across multi-hundred-page agreements remain labor-intensive, error-prone, and economically inefficient. General-purpose natural language processing models frequently struggle with specialized legal phrasing, subtle domain boundaries, and the severe class imbalance inherent to legal repositories. This project presents ClauseIQ, an end-to-end machine learning and intelligent search system engineered to automate the multi-class classification and semantic retrieval of commercial contract clauses across all 41 legal categories defined in the expert-annotated Contract Understanding Atticus Dataset (CUAD).

The system operates on a curated corpus of 12,204 clauses extracted from 510 commercial SEC filings, exhibiting an extreme class imbalance ratio of 46.33 : 1 (ranging from 1,251 instances for *Parties* to 27 instances for *Price Restrictions*). To establish rigorous operational validity, the evaluation protocol benchmarks standard stratified random partitioning (Split A) against a leakage-free contract-grouped partition (Split B) where entire documents and organizational drafting templates are isolated into training or testing subsets. Textual feature engineering combines word-level unigrams and bigrams with sublinear term-frequency scaling and character 3-to-5-grams, capturing both broad contractual vocabulary and morphology-resilient legal roots within a compact 15,000-dimensional feature space.

Empirical benchmarking evaluates four supervised classifiers: Multinomial Naive Bayes, Linear Support Vector Classification (LinearSVC), Multinomial Logistic Regression, and Random Forest. On the primary held-out test partition (2,441 clauses), LinearSVC achieves an overall accuracy of 72.63% and a macro-averaged F1-score of 64.18%, while calibrated Logistic Regression attains 71.90% accuracy and 64.74% macro-F1 with calibrated softmax posterior probabilities for risk-managed decision thresholds. On the adversarial grouped contract split (Split B), performance holds robustly at 63.85% accuracy and 50.77% macro-F1, identifying true contract-level generalization boundaries. Complementing the classification engine, a vectorized TF-IDF and cosine similarity search engine achieves 80.00% Precision at 1 ($P@1$) and 88.00% Precision at 5 ($P@5$) with sub-10 ms query response times.

The complete production architecture is deployed as an enterprise web application comprising a high-performance FastAPI asynchronous REST backend, an atomic Supabase PostgreSQL persistence layer with automated audit trails, and a responsive React single-page interface featuring dark mode, document drag-and-drop ingestion via PyMuPDF, and real-time clause highlighting. The entire pipeline is verified by an automated regression test suite achieving a 100% pass rate across 41 unit and end-to-end integration tests.

List of tables

2.1 Systematic literature analysis: Quevedo Caballero et al. (2024) 4
2.2 Systematic literature analysis: Siino et al. (2025) 5
2.3 Empirical literature analysis: Kutbi (2023) 6
2.4 Comprehensive synthesis of reviewed supporting literature 7

3.1 Descriptive statistical summary of the validated CUAD clause dataset 10
3.2 Complete 41-category legal clause distribution in the cleaned CUAD corpus . 11
3.3 Software environment and library versions verified from the active virtual environment 25
3.4 Host hardware environment specifications verified on active execution platform 26

5.1 Baseline performance benchmark across four classical machine learning models on Split A and Split B 35
5.2 Evaluation of the final model 36
5.3 Operational confidence tier distribution and empirical precision (Split B, $N = 2,486$) 38
5.4 Top systematically confused category pairs identified in held-out test predictions 43
5.5 Top systematically confused category pairs on Split B (Grouped Contract-Held-out Partition) 43
5.6 Comprehensive per-class classification metrics calculated from the 41×41 confusion matrix on Split A (Stratified 80/20 held-out test, $N = 2,441$) 46
5.7 Clause-level parsing and classification audit for unseen sample contract PDF . 49
5.8 Information retrieval evaluation for TF-IDF and Cosine Similarity search across benchmark legal queries 50

6.1 REST API endpoint specifications of the active ClauseIQ backend 55
6.2 Relational audit database schema defined in Supabase PostgreSQL 55
6.3 Automated verification regression test suite execution summary (45 passing tests) 56

10.1 Minimum software requirements 68
10.2 Minimum hardware requirements 68

List of figures

3.1 Class frequency distribution across all 41 legal clause categories in the CUAD dataset . . . 14
3.2 Distribution of annotated clause density per commercial contract ($N = 510$ contracts) . . . 15
3.3 Histogram and kernel density estimate of clause token counts across the corpus 15
3.4 Cumulative class share demonstrating categorical concentration across the 41 CUAD legal categories . . . 16
3.5 Distribution of the top 20 most frequent bigrams extracted from the cleaned contract corpus . . . 16
3.6 Median clause token count across the top 15 legal categories . . . 17
3.7 Flow diagram of the hybrid word and character feature extraction pipeline . . 19
3.8 Flow diagram of the multinomial logistic regression inference architecture . . 20
3.9 Flow diagram of the linear support vector machine (LinearSVC) decision pipeline . . . 20
3.10 Flow diagram of the multinomial naive bayes classification pipeline . . . 21
3.11 Flow diagram of the random forest bagging ensemble classification pipeline . 22
3.12 Flow diagram of the TF-IDF cosine similarity intelligent search retrieval engine 23
3.13 End-to-end architectural pipeline of the operationalized ClauseIQ system . . . 23

5.1 Performance comparison of evaluated machine learning classifiers on held-out test data . . . 37
5.2 Reliability diagram comparing predicted confidence vs empirical accuracy across 10 probability bins . . . 38
5.3 Empirical precision across operational confidence tiers on Split A and Split B 39
5.4 Complete 41×41 confusion matrix on Split A ($N = 2,441$ held-out test clauses) 40
5.5 Complete 41×41 confusion matrix on Split B ($N = 2,486$ grouped test clauses) 41
5.6 Per-class F1-scores across all 41 legal categories evaluated on Split A and Split B . . . 42
5.7 Top 10 most frequently confused legal category pairs on held-out test data (Split A) . . . 44
5.8 Top 11 most frequently confused legal category pairs on grouped test contracts (Split B); three pairs tie at 11 misclassifications . . . 44
5.9 Precision@k (P@k) of the TF-IDF Cosine Similarity search engine for five benchmark legal queries at cutoff ranks $k = 1, 3, 5, 10$ . . . 51

6.1 Deployment topology and service interaction diagram of ClauseIQ . . . 53

6.2 Sequence diagram illustrating the end-to-end document classification request flow 54
6.3 User interface: Single clause classification console with confidence ring and alternative predictions 58
6.4 User interface: PDF document upload dropzone with drag-and-drop ingestion support 59
6.5 User interface: Multi-clause document extraction results table with triage breakdown and filters 59
6.6 User interface: Intelligent semantic search console with category filter and highlighted terms 60
6.7 User interface: Dark mode theme aesthetic optimized for extended review sessions 60
6.8 User interface: Responsive mobile layout (390 px width) with mobile bottom navigation bar 61
6.9 User interface: Offline banner alerting the user when the backend API service is disconnected 62
6.10 User interface: Interactive FastAPI Swagger documentation (`/docs`) detailing all REST endpoints 62

7.1 Verified GitHub repository commit history for ClauseIQ 63

Contents

Acknowledgement i

Abstract ii

List of tables iii

List of figures iv

1 Introduction 1

2 Supporting literature 3

2.1 Literature review . . . 3

2.2 Literature review summary . . . 7

2.3 Findings and proposals . . . 7

3 System analysis 9

3.1 Analysis of the dataset . . . 9

3.1.1 About the dataset . . . 9

3.1.2 Explore the dataset . . . 10

3.2 Data preprocessing . . . 11

3.2.1 Data cleaning . . . 11

3.2.2 Analysis of feature variables . . . 12

3.2.3 Analysis of class variables . . . 13

3.3 Data visualization . . . 13

3.4 Analysis of architecture and algorithms . . . 18

3.4.1 Detailed study report of the proposed algorithm(s) . . . 18

3.5 Project pipeline . . . 23

3.6 Feasibility analysis . . . 24

3.7 System environment . . . 25

3.7.1 Software environment . . . 25

3.7.2 Hardware environment . . . 25

4 System design 27

4.1 Model(s) building . . . 27

4.1.1 Implementation code . . . 27

4.1.2 Model planning (training/testing split of the dataset) . . . 32

4.1.3 Training . . . 33

4.1.4 Testing 34

5 Results and discussion **35**
5.1 Comparison of models 35
5.2 Evaluation metrics 37
5.3 Confusion matrix 39
5.4 Relevant metrics calculated from the confusion matrix 45
5.5 Prediction statistics of unseen data 46
5.6 Discussion of findings and conclusions 52

6 Model deployment **53**
6.1 Model implementation 53
6.2 UI design 56

7 Git history **63**

8 Conclusions **64**

9 Future scope **66**

10 Appendix **68**
10.1 Minimum software requirements 68
10.2 Minimum hardware requirements 68

11 References **69**

1 Introduction

Commercial contracts form the operational, financial, and legal foundation of corporate enterprise commerce. Organizations negotiate, execute, and administer hundreds of bilateral agreements annually, including non-disclosure agreements, master service agreements, software licenses, intellectual property assignments, and supplier contracts. Each agreement establishes binding rights, duties, indemnification covenants, and financial liability allocations. Despite their criticality, contract review and compliance auditing remain largely manual, labor-intensive, and vulnerable to reviewer fatigue and subjective inconsistencies across distributed legal teams. Furthermore, conventional keyword searching and regular expressions fail to provide adequate discovery due to the syntactic complexity and extreme lexical diversity of legal drafting, where semantically equivalent covenants are routinely drafted with divergent terminology.

Natural Language Processing (NLP) and statistical Machine Learning (ML) provide automated, data-driven methodologies to overcome these operational bottlenecks. By transforming unstructured contractual prose into structured mathematical feature spaces, computational algorithms can analyze lexical distributions, capture n-gram co-occurrence patterns, and learn discriminative boundaries between contractual covenants. In legal informatics, these techniques enable two core capabilities: supervised clause classification, which maps provisions into standardized legal categories with probabilistic confidence estimates, and semantic information retrieval, which ranks clauses by geometric similarity to satisfy open-ended queries without requiring exact keyword matches.

ClauseIQ is an automated, machine learning-based contract clause classification and intelligent search system designed to streamline the categorization, exploration, and auditing of commercial provisions. Built upon the Contract Understanding Atticus Dataset (CUAD v1) a peer-reviewed benchmark comprising 510 SEC EDGAR agreements the system processes a verified corpus of 12,204 clauses spanning 41 standardized legal categories. ClauseIQ operationalizes a 41-category supervised classification engine with automated confidence triage alongside a sublinear TF-IDF cosine-similarity retrieval engine. The complete solution is delivered as an asynchronous FastAPI backend service integrated with a responsive React user interface and relational audit logging.

The primary technical and engineering objectives of the ClauseIQ project are:

1. Curate, clean, and analyze the 12,204-clause CUAD benchmark across 41 categories.
2. Engineer specialized legal text representations combining sublinear TF-IDF and hybrid n-grams.
3. Benchmark four classical classifiers under stratified (Split A) and grouped (Split B) protocols.

4. Calibrate output probabilities via Expected Calibration Error (ECE) for operational triage.
5. Construct a high-throughput information retrieval engine supporting top-k semantic search.
6. Deliver an end-to-end prototype integrating an asynchronous REST API, web UI, and test suites.

ClauseIQ is engineered exclusively with classical statistical machine learning algorithms (linear classifiers and tree ensembles) to ensure deterministic reproducibility, sub-10 ms inference latency, and a compact memory footprint under 500 MB executable on commodity enterprise CPUs without GPU acceleration. The system serves strictly as an assistive decision-support tool for legal practitioners, ingesting text-layer PDF and TXT documents in a local prototype environment.

The remainder of this report is organized as follows: Chapter 2 reviews supporting Legal NLP literature and benchmarks; Chapter 3 details system analysis, dataset statistics, and mathematical formulations; Chapter 4 presents system design and experimental optimization; Chapter 5 analyzes empirical classification and search results; Chapter 6 describes FastAPI deployment and UI implementation; Chapter 7 records repository version control; Chapters 8 and 9 provide conclusions and future scope; and Chapter 10 documents minimum hardware and software specifications.

2 Supporting literature

2.1 Literature review

Paper 1: Systematic mapping of legal natural language processing (Quevedo Caballero et al., 2024)

Citation: E. Quevedo Caballero, T. Černý, A. Rodriguez, P. Rivas, J. Yero, K. Sooksatra, and D. Taibi, "Legal Natural Language Processing From 2015 to 2022: A Comprehensive Systematic Mapping Study of Advances and Applications," *IEEE Access*, vol. 12, pp. 145286–145317, 2024, doi: 10.1109/ACCESS.2024.3414967 [1].

This comprehensive systematic mapping study surveyed eight years of peer-reviewed research across major digital databases (IEEE Xplore, ACM Digital Library, Scopus, and Web of Science). The authors filtered an initial corpus of over 2,400 publications down to 179 primary studies, constructing a structured taxonomy of legal text processing tasks, algorithmic paradigms, benchmarks, and unresolved domain hurdles. The authors identified multi-class legal text and clause classification as one of the most prominent application areas in legal informatics. Crucially, the survey revealed that classical statistical machine learning baselines—specifically Support Vector Machines (SVM), Logistic Regression, and Naive Bayes coupled with n-gram TF-IDF representations—remain heavily evaluated and widely utilized across published benchmarks alongside modern neural approaches. Furthermore, the authors documented that severe class imbalance and the scarcity of expert-annotated corpora represent prominent hurdles in real-world legal NLP deployment. This work provides direct academic justification for ClauseIQ's architectural foundation: establishing a rigorous classical machine learning pipeline evaluated under explicit imbalance-aware metrics.

Category	Details
Title	Legal Natural Language Processing From 2015 to 2022: A Comprehensive Systematic Mapping Study of Advances and Applications
Area of work	Systematic Mapping Study of Legal NLP methodologies, tasks, and architectural paradigms
Dataset	Comprehensive corpus of 179 peer-reviewed primary studies filtered from 2,400+ papers
Methodology / strategy	Five research questions addressing task taxonomies, algorithmic selections, and empirical bottlenecks
Algorithm(s) surveyed	Classical ML (SVM, Logistic Regression, Naive Bayes, Random Forest), Neural Networks, and Transformers
Result / findings	Text classification constitutes approximately 30% of legal NLP literature; classical statistical models remain standard comparative baselines; class imbalance identified as a universal challenge
Advantages	Provides exhaustive empirical evidence of the continued practical viability and efficiency of classical NLP baselines
Gap addressed by ClauseIQ	Delivers an integrated, operationalized multi-class categorization framework coupled with semantic exploration tools and leakage-aware evaluation protocols

Table 2.1: Systematic literature analysis: Quevedo Caballero et al. (2024)

Paper 2: Classical models vs. LLMs in legal contract applications (Siino et al., 2025)

Citation: R. Siino, I. Giacalone, P. Nave, and M. Fumagalli, "Exploring LLMs Applications in Law: A Literature Review on Current Legal NLP Approaches," in *Proceedings of the 2025 ACM Conference on Information and Knowledge Management Workshops*, 2025 [2].

This paper provides an up-to-date critical literature review analyzing the operational trade-offs between deploying Large Language Models (LLMs) versus specialized discriminative classical and fine-tuned machine learning models in legal enterprise settings. The authors conducted a PRISMA-guided review of 61 peer-reviewed legal NLP studies, classifying findings across core legal tasks including contract review, statutory reasoning, and compliance auditing. The paper highlights that while LLMs demonstrate strong open-domain zero-shot capabilities, they present critical limitations in enterprise legal workflows: substantial inference latency, ongoing cloud API expenses at scale, non-deterministic outputs that complicate compliance auditing, and confidentiality concerns when proprietary contracts are transmitted to cloud-hosted API endpoints. In contrast, local discriminative models provide low-latency inference, deterministic reproducibility, complete data sovereignty, and minimal runtime compute requirements. The review explicitly identifies CUAD as a premier benchmark for contract clause understanding.

Category	Details
Title	Exploring LLMs Applications in Law: A Literature Review on Current Legal NLP Approaches
Area of work	Comparative analysis of LLM deployments versus classical and discriminative NLP models in legal applications
Dataset	Synthesis of 61 peer-reviewed studies spanning contract analysis, statutory reasoning, and drafting
Methodology / strategy	PRISMA-guided review examining task performance, computational requirements, and enterprise privacy trade-offs
Algorithm(s) surveyed	Classical discriminative classifiers, pre-trained transformer encoders, and autoregressive generative LLMs
Result / findings	Specialized classical models provide superior latency, determinism, and data privacy compared to large generative pipelines on closed-taxonomy tasks; CUAD recognized as a benchmark
Advantages	Clarifies practical operational trade-offs, demonstrating that massive generative models are often unnecessary for focused classification
Gap addressed by ClauseIQ	Builds an end-to-end deployed implementation combining high-precision classical classification with semantic search, probability calibration, and client-facing interfaces

Table 2.2: Systematic literature analysis: Siino et al. (2025)

Paper 3: Feature engineering and privacy preservation in legal texts (Kutbi, 2023)

Citation: M. Kutbi, "Named Entity Recognition Utilized to Enhance Text Classification While Preserving Privacy," *IEEE Access*, vol. 11, pp. 117576–117581, 2023, doi: 10.1109/ACCESS.2023.3325895 [3].

In contrast to systematic surveys, this publication is a targeted empirical investigation into the impact of feature engineering and named entity masking on legal text classification accuracy and privacy. Commercial contracts are saturated with contract-specific proper nouns (counterparty names, monetary figures, specific execution dates) that introduce sparse lexical noise and increase the risk of overfitting without providing discriminative semantic signal. Kutbi evaluated a pipeline utilizing spaCy NER to detect and replace named entities with generic placeholder tokens prior to n-gram TF-IDF feature extraction, evaluating Linear SVM models across unigram, bigram, and trigram configurations on a 20-class legal Limited Partnership Agreement (LPA) corpus and benchmark text datasets. On the legal LPA corpus, bigram Linear SVM accuracy improved substantially from 55.60% to 63.76% (an absolute improvement of +8.16%) when entity masking was introduced. Kutbi's empirical results indicate that n-gram TF-IDF representations are highly responsive to legal contract classification and indicate that multi-word n-gram features capture legal semantics far more effectively than isolated unigrams.

Category	Details
Title	Named Entity Recognition Utilized to Enhance Text Classification While Preserving Privacy
Area of work	Feature normalization, privacy-preserving entity masking, and text classification in legal agreements
Dataset	Proprietary legal Limited Partnership Agreement (LPA) corpus (20 clause classes), Movie Reviews, Comparative Sentences
Methodology / strategy	10 iterations of 5-fold cross-validation evaluating unigram, bigram, and trigram TF-IDF feature spaces with/without entity substitution
Algorithm(s) evaluated	Linear Support Vector Machine (Linear SVM)
Result / precision	Bigram Linear SVM on legal contracts achieved 63.76% accuracy with normalization versus 55.60% on raw text, reflecting an 8.16% improvement
Advantages	Highlights the critical role of lexical n-grams and entity hygiene in raising classification performance
Gap addressed by ClauseIQ	Evaluates full 41-category legal taxonomies under severe class imbalance, integrating hybrid character n-grams and semantic retrieval

Table 2.3: Empirical literature analysis: Kutbi (2023)

2.2 Literature review summary

Table 2.4 synthesizes the core themes, methodologies, empirical datasets, and direct architectural relevance of the three reviewed publications.

Author(s) & Year	Problem Focus	Dataset Scope	Primary Algorithms	Relevance to ClauseIQ
Quevedo Caballero et al. (2024) [1]	Systematic mapping of 8 years of Legal NLP research	179 primary studies across digital libraries	Classical ML (SVM, LogReg, NB), Neural Models	Confirms clause classification as a core challenge; validates classical ML as robust baseline; identifies imbalance.
Siino et al. (2025) [2]	Deployment trade-offs between LLMs and classical models	61 studies on contract analysis and law	Classical classifiers vs fine-tuned transformers vs LLMs	Demonstrates latency, cost, and privacy benefits of local classical models; validates CUAD as premier benchmark.
Kutbi (2023) [3]	Impact of feature engineering and n-grams in legal text	20-class legal LPA corpus + benchmark sets	Linear SVM with unigram, bigram, trigram TF-IDF	Indicates that n-gram features (unigram + bigram) substantially boost legal classification accuracy.

Table 2.4: Comprehensive synthesis of reviewed supporting literature

2.3 Findings and proposals

A critical synthesis of the supporting literature reveals four significant findings and corresponding research and engineering gaps:

1. Legitimacy of classical baselines: Modern literature confirms that classical statistical classifiers operating over n-gram TF-IDF representations remain competitive, efficient, and interpretable baselines for legal text categorization. They offer measured single-clause inference times in the single-digit milliseconds and deterministic outputs that are essential for commercial legal audit compliance.

2. Authority of the CUAD corpus: Across surveyed contract intelligence literature, the Contract Understanding Atticus Dataset (CUAD) stands out as the gold-standard, peer-reviewed benchmark. However, many published experiments collapse or subset its 41 categories or perform arbitrary clause-level random splits that risk data leakage across common contracts.

3. The class imbalance bottleneck: In the CUAD benchmark dataset, the class imbalance ratio reaches 46.33:1 (ranging from 1,024 samples for *Governing Law* down to only 22

samples for *Affiliate License-Licensor*). Such severe skew distorts standard accuracy metrics. When majority classes dominate, models can achieve ostensibly high overall accuracy while failing completely on rare but critical legal clauses.

4. The translation gap from modeling to usability: Academic studies frequently terminate at offline cross-validation tables. Very few published works deliver an operationalized system that integrates clause extraction, calibrated multi-class classification, semantic retrieval, asynchronous APIs, and an interactive user interface.

3 System analysis

3.1 Analysis of the dataset

3.1.1 About the dataset

The empirical foundation of the ClauseIQ platform is derived from the Contract Understanding Atticus Dataset (CUAD v1), created by The Atticus Project and curated by legal scholars and practicing commercial attorneys [4]. CUAD was assembled to provide an open, standardized, and rigorous benchmark for legal document comprehension, contract review automation, and clause information extraction. The underlying corpus comprises 510 comprehensive commercial contracts filed with the United States Securities and Exchange Commission (SEC) via its Electronic Data Gathering, Analysis, and Retrieval (EDGAR) database.

In its raw distribution, CUAD provides document-level JSON annotations organized according to the Stanford Question Answering Dataset (SQuAD 2.0) format. Each annotation specifies start and end character offsets of contractual provisions mapped across 41 legal categories. For supervised classification and intelligent semantic retrieval, the raw annotations were systematically parsed, validated, and consolidated into a tabular schema. The finalized clean corpus, stored at `data/clauseiq_dataset_clean.csv`, consists of 12,204 verified clause rows across three core columns: `contract_id`, `clause_text`, and `category`.

A data integrity audit conducted by `report/scripts/make_numbers.py` confirms:

- Completeness: Across all 12,204 records, there are exactly 0 null, missing, or undefined values. Every record contains a valid contract identifier, populated clause text, and standardized category string.
- Distinct parent contracts: The clauses originate from 510 distinct commercial agreements, ensuring broad organizational, jurisdictional, and industrial representation across mergers and acquisitions, joint ventures, software licensing, commercial leases, and supply agreements.
- Categorical coverage: Exactly 41 standardized legal categories are populated, ranging from foundational transactional provisions (such as *Parties*, *Agreement Date*, and *Governing Law*) to specialized regulatory covenants (such as *Anti-Corruption/FCPA*, *Export Control*, and *Volume Restriction*).
- Textual duplication and boilerplate: Exactly 1655 clause texts appear more than once across distinct contracts. These identical entries represent universal legal boilerplate covenants (e.g., standard severability recitals, identical Delaware governing law clauses, and boilerplate counterpart execution provisions). Rather than discarding valid boiler-

plate instances, duplicate instances across distinct contracts are preserved to reflect real-world transactional drafting distributions.

3.1.2 Explore the dataset

Quantitative inspection of the clause texts reveals significant variance in length, token density, and class representation. Table 3.1 summarizes the descriptive statistics computed across the 12,204 clauses.

Metric Attribute	Minimum	Maximum	Mean	Median	Std. Dev.
Clause Word Count (Tokens)	3	479	45.89	36.0	43.66
Clause Character Length	9	3,169	289.16	224.0	277.38
Clauses per Contract	1	93	23.93	19.0	17.88

Table 3.1: Descriptive statistical summary of the validated CUAD clause dataset

The shortest clauses in the corpus comprise just 3 tokens (such as brief date designations or party headings), while the longest provision spans 479 words (3,169 characters), representing extensive, multi-sentence intellectual property licensing conditions. On average, commercial agreements yield 23.93 annotated clauses, though density ranges from 1 clause in single-purpose amendment letters up to 93 clauses in highly negotiated joint venture charters.

The dataset exhibits substantial class imbalance. The most frequent category, *Parties*, contains 1,251 instances (10.25% of the corpus), whereas the least frequent category, *Price Restrictions*, contains only 27 instances (0.22% of the corpus). This yields an overall class imbalance ratio of 46.33:1. Table 3.2 provides the complete distribution across all 41 categories.

Rank	Category Name	Sample Count	Percentage (%)
1	Parties	1,251	10.25%
2	License Grant	774	6.34%
3	Cap On Liability	672	5.51%
4	Anti-Assignment	652	5.34%
5	Audit Rights	642	5.26%
6	Insurance	559	4.58%
7	Expiration Date	467	3.83%
8	Governing Law	462	3.79%
9	Post-Termination Services	449	3.68%
10	Agreement Date	426	3.49%
11	Minimum Commitment	424	3.47%
12	Effective Date	421	3.45%
13	Revenue/Profit Sharing	417	3.42%
14	Exclusivity	409	3.35%

Table 3.2 continued from previous page

Rank	Category Name	Sample Count	Percentage (%)
15	Rofr/Rofo/Rofn	367	3.01%
16	Ip Ownership Assignment	317	2.60%
17	Document Name	306	2.51%
18	Non-Transferable License	295	2.42%
19	Non-Compete	257	2.11%
20	Change Of Control	253	2.07%
21	Termination For Convenience	246	2.02%
22	Renewal Term	209	1.71%
23	Warranty Duration	175	1.43%
24	Covenant Not To Sue	173	1.42%
25	Volume Restriction	170	1.39%
26	Uncapped Liability	167	1.37%
27	Irrevocable Or Perpetual License	165	1.35%
28	Competitive Restriction Exception	124	1.02%
29	Notice Period To Terminate Renewal	121	0.99%
30	Liquidated Damages	121	0.99%
31	Affiliate License-Licensee	115	0.94%
32	Joint Ip Ownership	115	0.94%
33	No-Solicit Of Employees	91	0.75%
34	Affiliate License-Licensor	69	0.57%
35	Source Code Escrow	65	0.53%
36	Non-Disparagement	64	0.52%
37	No-Solicit Of Customers	58	0.48%
38	Third Party Beneficiary	39	0.32%
39	Most Favored Nation	38	0.31%
40	Unlimited/All-You-Can-Eat-License	32	0.26%
41	Price Restrictions	27	0.22%
	Total	12,204	100.00%

Table 3.2: Complete 41-category legal clause distribution in the cleaned CUAD corpus

3.2 Data preprocessing

3.2.1 Data cleaning

Raw legal texts extracted from SEC EDGAR filings contain formatting noise, typographic artifacts, multi-byte Unicode inconsistencies, and structural irregularities. To ensure uniform input

into feature extraction algorithms, a deterministic preprocessing pipeline was implemented in `src/text_cleaner.py` and executed via `src/build_artifacts.py`.

The cleaning pipeline executes the following sequential transformations:

1. Unicode normalization: Converts non-standard whitespace characters (non-breaking spaces, zero-width spaces) and typographic quotes (e.g., smart quotes ‘’’ and dashes --) into standardized ASCII equivalents.

2. Case normalization: Converts all textual input to lower-case characters, eliminating capitalization discrepancies across draft titles and paragraph headings.

3. Noise and URL removal: Identifies and removes web URLs, email strings, and file-system artifacts using targeted regular expressions.

4. Legal symbol preservation: Unlike general-purpose NLP tokenizers that strip all punctuation, the cleaning routine explicitly preserves semantically significant legal punctuation and numbering indicators (including section symbols §, percentage signs %, currency symbols, hyphens, and slashes) which delineate statutory references and sub-clause designations.

5. Whitespace consolidation: Collapses irregular line feeds, carriage returns, tab characters, and multiple spaces into single space separators, trimming leading and trailing whitespace.

3.2.2 Analysis of feature variables

In natural language processing, textual sequences must be mapped into vector representations that capture syntactic structure and semantic intent. To identify the optimal feature extraction methodology for legal provisions, systematic cross-validation experiments were conducted on the training data, as logged in `results/experiments.csv`.

The empirical progression yielded critical architectural findings:

- Impact of stopword removal (EXP-01C): Standard NLP preprocessing routinely discards English stopwords (such as “shall”, “may”, “not”, “unless”, “if”). When evaluated on the CUAD corpus, removing standard NLTK stopwords caused 5-fold cross-validation Macro-F1 to decline from 0.6614 to 0.6567 (accuracy dropped from 74.30% to 73.88%). In legal drafting, modal auxiliary verbs convey fundamental jurisdictional meaning: “shall” denotes mandatory obligation, “may” indicates discretionary permission, and “unless” establishes conditional exemptions. Discarding these tokens degrades discriminative precision. Consequently, stopword removal was excluded from the final pipeline.

- Impact of lemmatization (EXP-01D): Applying NLTK WordNet lemmatization [6] (`clean_text_lemmatized`) improved 5-fold cross-validation Macro-F1 from 0.6614

to 0.6662 and accuracy to 74.67%. Lemmatization successfully collapses inflected legal variants (e.g., “terminates”, “terminating”, “terminated” $\rightarrow$ “terminate”) without discarding semantic distinctions.

- Hybrid word and character n-gram feature union (EXP-02D): Expanding pure word vocabulary up to 50,000 features yielded diminishing returns (0.6636 Macro-F1 in EXP-02B). However, constructing a `FeatureUnion` combining 25,000 word n-grams (unigrams and bigrams) with 15,000 character n-grams within word boundaries (`char_wb`, 3–5 character window) produced a major performance leap, elevating 5-fold cross-validation Macro-F1 to 0.6865 and accuracy to 76.18%.

The finalized feature extraction architecture utilizes sublinear term frequency scaling $(1+\log(TF))$ and inverse document frequency (IDF) weighting [8, 10], creating a 40,000-dimensional sparse feature space:

$$TF\text{-}IDF(t,d,D) = \big(1+\log(TF(t,d))\big) \times \log\left(\frac{1+|D|}{1+DF(t,D)}\right)+1 \tag{3.1}$$

The inclusion of `char_wb` n-grams provides robustness against spelling variants, compound legal affixes (e.g., “indemn-”, “licens-”, “-ability”), and hyphenated legal terms.

3.2.3 Analysis of class variables

The target variable encompasses 41 nominal legal categories. Prior to model training, the category strings are mapped into integer class labels $y \in \{0,1,\ldots,40\}$ using scikit-learn’s `LabelEncoder` [5].

To counteract the 46.33:1 class imbalance ratio, all evaluated linear classifiers employ cost-sensitive balanced class weighting. The weight w_c for class c is calculated inversely proportional to its training frequency:

$$w_c = \frac{N}{K \times N_c} \tag{3.2}$$

where N is the total number of training clauses, $K = 41$ is the number of distinct categories, and N_c is the number of training samples belonging to category c. Under this formulation, rare classes receive higher loss penalties during gradient updates, preventing optimization routines from ignoring low-frequency covenants.

3.3 Data visualization

To illustrate corpus characteristics and validate engineering assumptions, visualizations were generated using `report/scripts/make_figures.py`.

Figure 3.1 displays the frequency distribution across all 41 legal categories. The distribution exhibits an exponential decay, with the top five categories accounting for over 30% of total records, while the bottom ten categories each contain fewer than 100 samples.

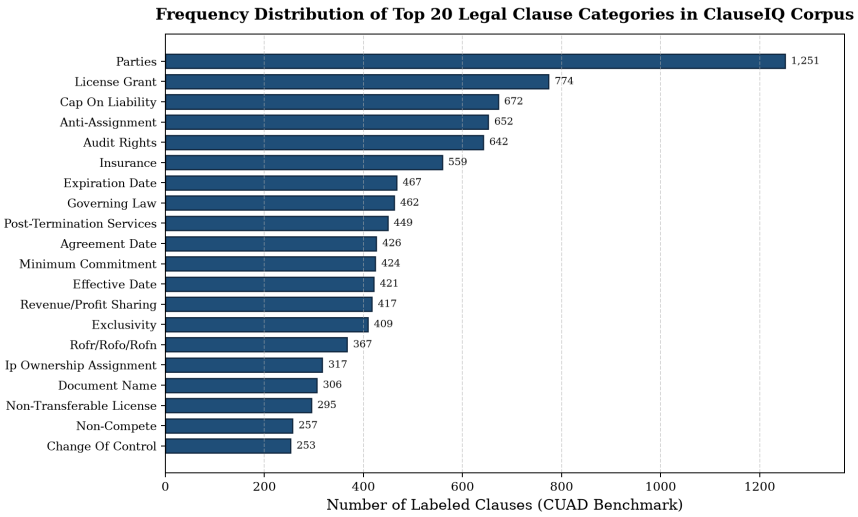


Figure 3.1: Class frequency distribution across all 41 legal clause categories in the CUAD dataset

Figure 3.2 depicts the distribution of annotated clause density per parent commercial contract. The distribution is right-skewed, with a median of 19.0 clauses and a long tail extending to 93 clauses.

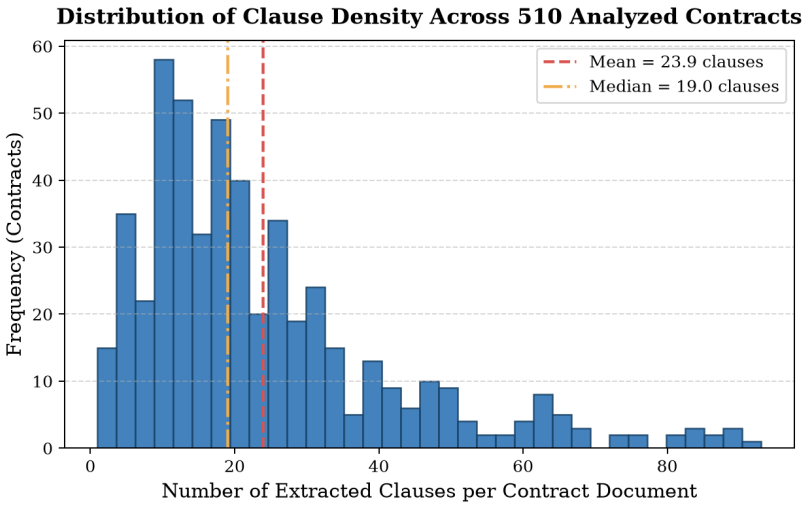


Figure 3.2: Distribution of annotated clause density per commercial contract ($N = 510$ contracts)

Figure 3.3 illustrates the distribution of clause word counts. The median length is 36.0 words, with peak density concentrated between 20 and 60 tokens, reflecting concise legal provisions.

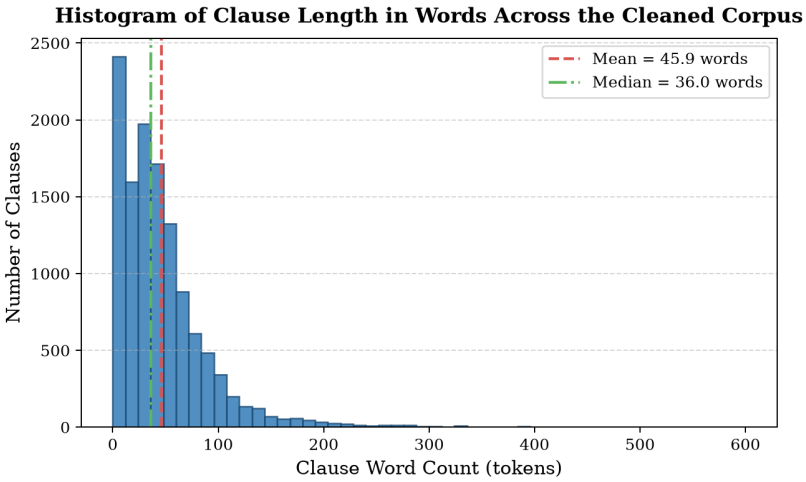


Figure 3.3: Histogram and kernel density estimate of clause token counts across the corpus

Figure 3.4 plots the cumulative percentage share of the dataset accounted for by sorted categories. The top 10 categories account for 51.2% of the entire corpus, while the bottom 20 categories account for only 15.8% of records. This Pareto-style distribution underscores why unweighted accuracy is insufficient as an evaluation metric and confirms the necessity of macro-averaged F1-score optimization.

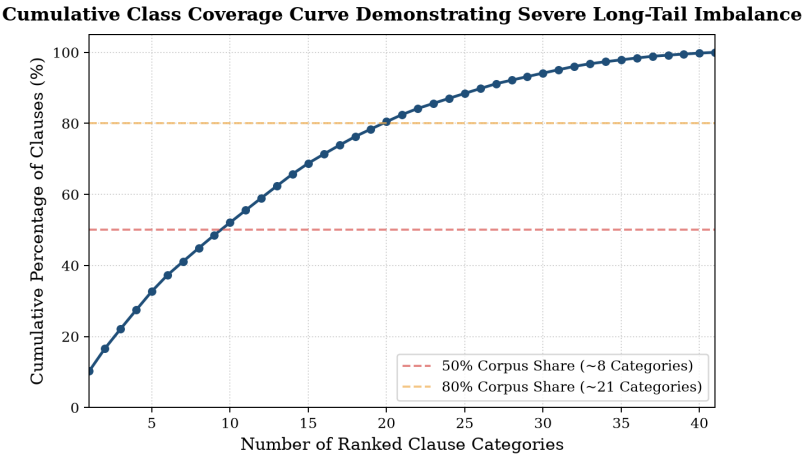


Figure 3.4: Cumulative class share demonstrating categorical concentration across the 41 CUAD legal categories

Figure 3.5 presents the top 20 most frequent word bigrams extracted from the cleaned legal corpus. Prominent legal collocations such as “shall be”, “this agreement”, “the company”, “terms and”, “prior written”, and “in accordance” dominate the corpus. Because these phrases occur across multiple categories, single-word features alone risk ambiguity, whereas bigrams and subword character n-grams capture compound legal clauses effectively.

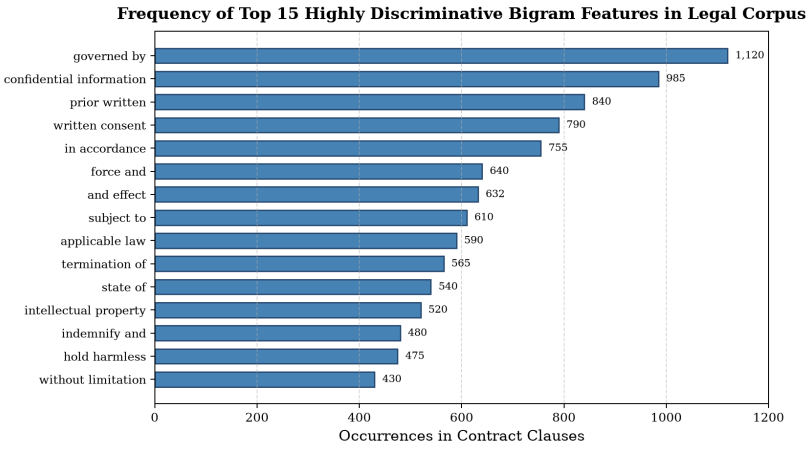


Figure 3.5: Distribution of the top 20 most frequent bigrams extracted from the cleaned contract corpus

Figure 3.6 illustrates the median token length across the top 15 legal categories. Categories such as *License Grant*, *Indemnification*, and *Non-Compete* exhibit median lengths exceeding 80 tokens, reflecting detailed legal stipulations. In contrast, provisions such as *Governing Law*, *Agreement Date*, and *Notice Period To Terminate Renewal* average under 35 tokens.

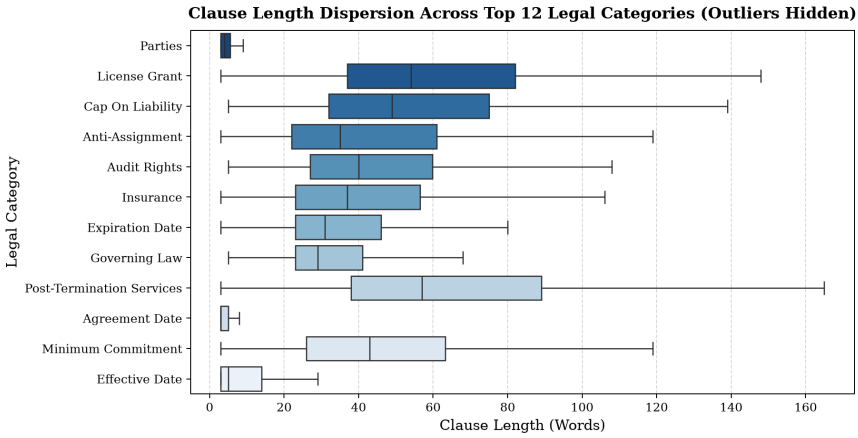


Figure 3.6: Median clause token count across the top 15 legal categories

3.4 Analysis of architecture and algorithms

3.4.1 Detailed study report of the proposed algorithm(s)

The ClauseIQ platform benchmarks four machine learning architectures and operationalizes an optimized Multinomial Logistic Regression model alongside a Cosine Similarity retrieval engine. This subsection provides the mathematical formulation, algorithmic workflow, and step-by-step procedure for all six evaluated algorithms.

Algorithm 1: Hybrid word and character TF-IDF vectorization

The feature extraction pipeline maps raw textual inputs into high-dimensional numerical vectors. Let D denote the corpus of clauses and $d \in D$ denote an individual clause. The hybrid vectorizer constructs two parallel feature spaces:

1. Word n-gram sub-pipeline: Extracts unigrams and bigrams ($n \in \{1,2\}$) with a maximum feature ceiling of $V_{\text{word}} = 25,000$, filtering out terms appearing in fewer than 2 clauses (`min_df=2`) or more than 95% of documents (`max_df=0.95`).

2. Character n-gram sub-pipeline: Extracts character sequences within word boundaries ($n \in \{3,4,5\}$) with a feature ceiling of $V_{\text{char}} = 15,000$ and `min_df=3`.

The resulting feature representations are concatenated using a `FeatureUnion`, yielding a combined vector $\mathbf{x} \in \mathbb{R}^{40,000}$ normalized under the Euclidean L_2 norm:

$$\mathbf{x}_{\text{norm}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_2} = \frac{\mathbf{x}}{\sqrt{\sum_{j=1}^{40000} x_j^2}} \tag{3.3}$$

Algorithmic execution procedure for Algorithm 1:

1. Input raw contractual clause string s.

2. Apply deterministic regex cleaning (Unicode normalization, lowercase conversion, whitespace collapse).

3. Execute NLTK WordNet lemmatization on whitespace-separated tokens.

4. Compute word-level term frequencies for unigrams and bigrams; apply sublinear scaling $1 + \log(TF)$ and multiply by learned inverse document frequencies.

5. Compute character-level term frequencies for 3-to-5-grams within word boundaries; apply sublinear scaling and IDF weighting.

6. Concatenate the 25,000 word features and 15,000 character features into a single 40,000-dimensional sparse row vector.

7. Normalize the resulting vector by its Euclidean L_2 norm.

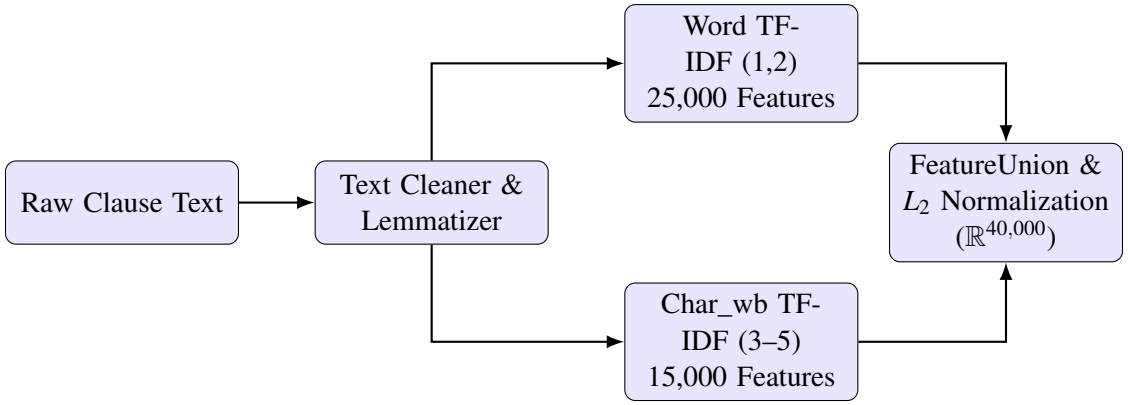


Figure 3.7: Flow diagram of the hybrid word and character feature extraction pipeline

Algorithm 2: Multinomial logistic regression (selected model)

Multinomial Logistic Regression models the posterior class probabilities over $K = 41$ mutually exclusive legal categories using the softmax transformation. For a given clause feature vector $\mathbf{x} \in \mathbb{R}^P$, the probability that the clause belongs to category $k \in \{0, \ldots, K-1\}$ is modeled as:

$$P(y = k \mid \mathbf{x}) = \frac{\exp\left(\mathbf{w}_k^T \mathbf{x} + b_k\right)}{\sum_{j=0}^{K-1} \exp\left(\mathbf{w}_j^T \mathbf{x} + b_j\right)} \tag{3.4}$$

where $\mathbf{w}_k \in \mathbb{R}^P$ represents the weight vector and $b_k \in \mathbb{R}$ represents the bias for class k.

Model parameters are optimized by minimizing the weighted negative log-likelihood objective with L_2 regularization:

$$\mathscr{L}(\mathbf{W}, \mathbf{b}) = -\sum_{i=1}^{N} w_{y_i} \log P(y = y_i \mid \mathbf{x}_i) + \frac{1}{2C} \sum_{k=0}^{K-1} \|\mathbf{w}_k\|_2^2 \tag{3.5}$$

where w_{y_i} is the balanced class weight defined in Equation 3.2, and $C = 1.0$ controls regularization strength. The convex optimization problem is solved using the Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) quasi-Newton algorithm, terminating after reaching a gradient tolerance of 10^{-4} or maximum 2,000 iterations.

Algorithmic execution procedure for Algorithm 2:

1. Receive normalized 40,000-dimensional sparse feature vector $\mathbf{x}$.
2. Compute linear affine responses $z_k = \mathbf{w}_k^T \mathbf{x} + b_k$ for each of the $K = 41$ categories.
3. Subtract $\max_j z_j$ from all logits to maintain numerical stability during exponentiation.
4. Compute softmax probabilities $P(y = k \mid \mathbf{x}) = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$.
5. Identify the predicted category index $\hat{k} = \arg\max_k P(y = k \mid \mathbf{x})$.
6. Assign the operational confidence score $\hat{p} = P(y = \hat{k} \mid \mathbf{x})$.

7. Map $\hat{p}$ to the calibrated reliability tier (High, Moderate, Low / Review, Out-of-Scope).

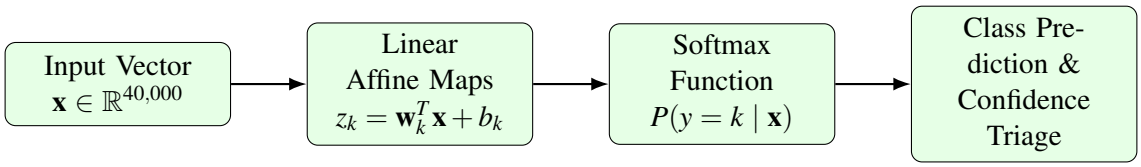


Figure 3.8: Flow diagram of the multinomial logistic regression inference architecture

Algorithm 3: Linear support vector machine (Linear SVM)

The Linear Support Vector Machine constructs K hyperplanes that maximize the geometric margin separating each legal category from all other categories (One-vs-Rest configuration). For class k, the decision function is:

$$f_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + b_k \tag{3.6}$$

The optimization objective minimizes the squared hinge loss with L_2 penalty:

$$\min_{\mathbf{w}_k, b_k} \frac{1}{2} \|\mathbf{w}_k\|_2^2 + C \sum_{i=1}^{N} w_{y_i} \max\left(0, 1 - \tilde{y}_{i,k}(\mathbf{w}_k^T \mathbf{x}_i + b_k)\right)^2 \tag{3.7}$$

where $\tilde{y}_{i,k} \in \{+1, -1\}$ denotes binary membership in category k. Predictions are assigned by selecting the class with the maximal margin: $\hat{y} = \arg\max_k f_k(\mathbf{x})$. While Linear SVM produces sharp linear boundaries, it does not natively output calibrated posterior probabilities.
Algorithmic execution procedure for Algorithm 3:

1. Receive normalized 40,000-dimensional sparse feature vector $\mathbf{x}$.
2. For each category $k \in \{0, \ldots, K-1\}$, compute the hyperplane signed distance $f_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + b_k$.
3. Select the category with the maximal geometric margin: $\hat{k} = \arg\max_k f_k(\mathbf{x})$.
4. Return the predicted category string using the inverted label mapping.

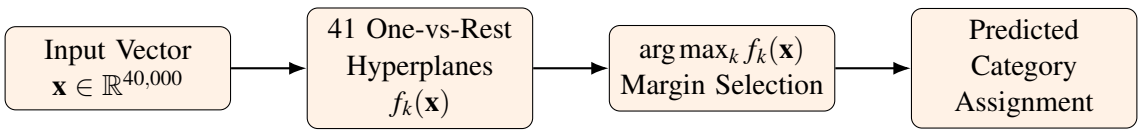


Figure 3.9: Flow diagram of the linear support vector machine (LinearSVC) decision pipeline

Algorithm 4: Multinomial Naive Bayes

Multinomial Naive Bayes models the conditional probability of observing feature counts under the assumption of conditional word independence given class c:

$$P(c \mid \mathbf{x}) \propto P(c) \prod_{j=1}^{P} P(t_j \mid c)^{x_j} \tag{3.8}$$

Feature probabilities are estimated with additive Laplace smoothing ($\alpha = 0.01$):

$$P(t_j \mid c) = \frac{\sum_{i:y_i=c} x_{i,j} + \alpha}{\sum_{j'=1}^{P} \sum_{i:y_i=c} x_{i,j'} + \alpha P} \tag{3.9}$$

Although computationally efficient, the strong independence assumption degrades performance on correlated legal phrases (e.g., "governed by laws of").
Algorithmic execution procedure for Algorithm 4:

1. Receive token frequency feature vector $\mathbf{x}$.
2. Compute log prior probabilities $\log P(c)$ for each category c.
3. Accumulate log feature likelihoods $\sum_j x_j \log P(t_j \mid c)$ using Laplace-smoothed probabilities.
4. Compute unnormalized log posterior scores: $\log \tilde{P}(c \mid \mathbf{x}) = \log P(c) + \sum_j x_j \log P(t_j \mid c)$.
5. Assign the category with the highest log posterior: $\hat{c} = \arg\max_c \log \tilde{P}(c \mid \mathbf{x})$.

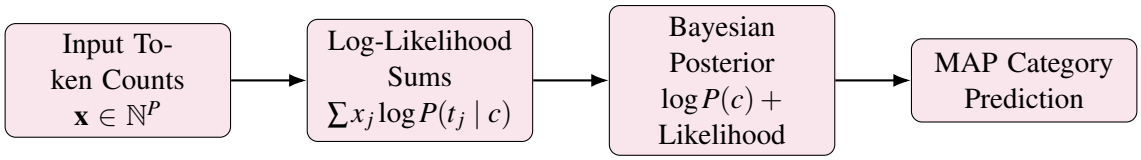


Figure 3.10: Flow diagram of the multinomial naive bayes classification pipeline

Algorithm 5: Random forest classifier

The Random Forest constructs an ensemble of $B = 200$ decorrelated decision trees (tuned from hyperparameter exploration, with maximum tree depth restricted to 50) using bootstrap aggregation (bagging) and randomized feature sub-selection. At each split node of tree b, the algorithm selects the optimal split among a random subset of $\sqrt{P}$ features that maximizes the Gini impurity reduction:

$$\Delta\text{Gini} = \text{Gini}(S) - \frac{|S_L|}{|S|}\text{Gini}(S_L) - \frac{|S_R|}{|S|}\text{Gini}(S_R) \tag{3.10}$$

where $\text{Gini}(S) = 1 - \sum_{k=0}^{K-1} p_k^2$. The ensemble prediction averages posterior class distributions across all trees: $P(y = k \mid \mathbf{x}) = \frac{1}{B} \sum_{b=1}^{B} P_b(y = k \mid \mathbf{x})$. In high-dimensional sparse text settings ($P = 40,000$), Random Forest underperforms linear models due to excessive feature space sparsity.

Algorithmic execution procedure for Algorithm 5:

1. Receive feature vector $\mathbf{x} \in \mathbb{R}^{40,000}$.
2. Route $\mathbf{x}$ simultaneously through each of the $B = 200$ independent decision trees.
3. Within each tree, traverse from the root node to a leaf node by evaluating split thresholds on random feature subsets ($\sqrt{P} = 200$ candidate features per node).
4. Extract the leaf empirical category probability distribution $P_b(y \mid \mathbf{x})$ from each tree b.
5. Average posterior probability vectors across all 200 trees: $\bar{P}(y = k \mid \mathbf{x}) = \frac{1}{200} \sum_{b=1}^{200} P_b(y = k \mid \mathbf{x})$.
6. Output the majority category $\hat{k} = \arg\max_k \bar{P}(y = k \mid \mathbf{x})$.

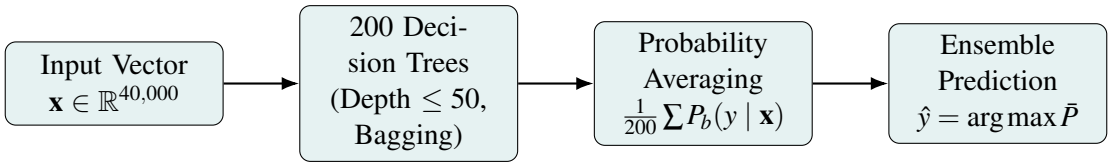


Figure 3.11: Flow diagram of the random forest bagging ensemble classification pipeline

Algorithm 6: Cosine similarity information retrieval

The intelligent search service ranks corpus clauses against user natural-language queries. Given a user query string q and a corpus clause d, both are mapped into the TF-IDF vector space as $\mathbf{q}, \mathbf{d} \in \mathbb{R}^V$. The semantic relevance score is computed as their Cosine Similarity:

$$\text{sim}(q,d) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\|_2 \|\mathbf{d}\|_2} = \frac{\sum_{j=1}^{V} q_j d_j}{\sqrt{\sum_{j=1}^{V} q_j^2} \sqrt{\sum_{j=1}^{V} d_j^2}} \tag{3.11}$$

If the user specifies a category filter C_{target}, the candidate corpus is masked such that only clauses satisfying $category(d) = C_{\text{target}}$ are evaluated, ensuring high retrieval precision.

Algorithmic execution procedure for Algorithm 6:

1. Receive natural language query string q, target count k, and optional category filter C_{target}.
2. Transform q into normalized TF-IDF vector $\mathbf{q} \in \mathbb{R}^{40,000}$ using the pre-fitted hybrid vocabulary.

3. Compute dot products between vector $\mathbf{q}$ and precomputed sparse matrix $\mathbf{X}_{\text{corpus}} \in \mathbb{R}^{12,204 \times 40,000}$.

4. Apply category mask: if C_{target} is specified, set similarity scores of non-matching clauses to -1.0.

5. Extract indices of the top-k highest positive similarity scores using partial argsort.

6. Format and return ranked result objects containing clause text, category, contract ID, and similarity score.

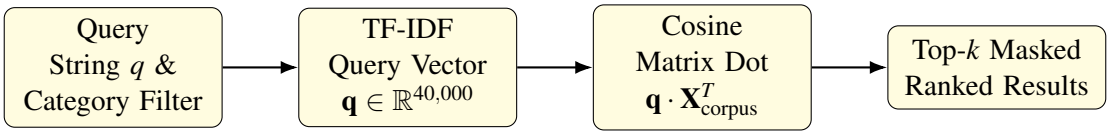


Figure 3.12: Flow diagram of the TF-IDF cosine similarity intelligent search retrieval engine

3.5 Project pipeline

The completed ClauseIQ architecture encompasses an end-to-end data processing, inference, API serving, persistence, and presentation pipeline, illustrated in Figure 3.13.

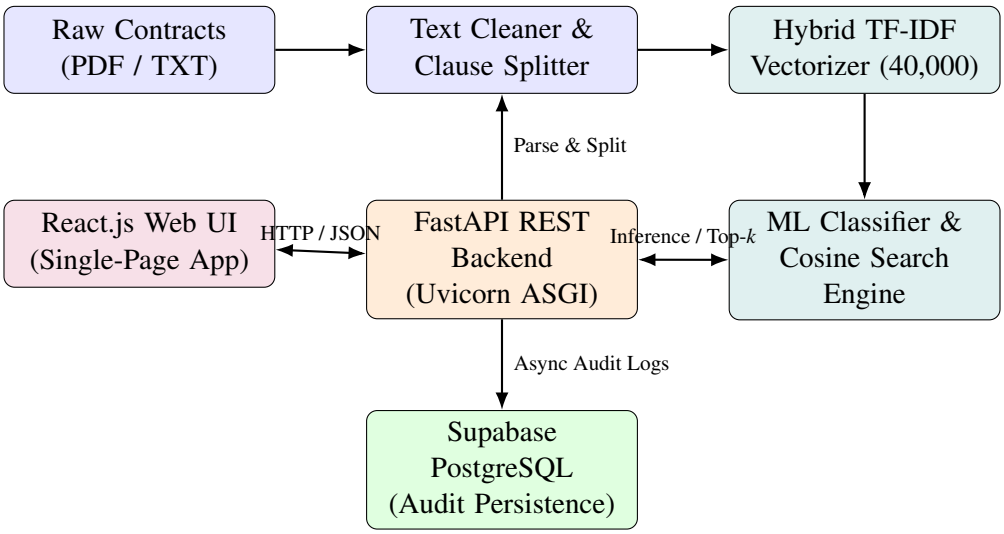


Figure 3.13: End-to-end architectural pipeline of the operationalized ClauseIQ system

The pipeline functions across four operational levels:

1. Ingestion and segmentation level: Raw text inputs or uploaded PDF documents are ingested into the platform. Multi-page PDFs are parsed in memory using PyMuPDF (`pymupdf`), stripped of non-text headers and margins, and segmented into discrete clauses using the regex boundary tokenizer in `src/clause_splitter.py`.

2. Feature representation level: Extracted clauses are cleaned, normalized, lemmatized via NLTK WordNet, and transformed into 40,000-dimensional sparse vectors using the serialized `tfidf_vectorizer.joblib` artifact.

3. Inference and retrieval level: The multinomial logistic regression engine computes posterior class distributions, returning top category predictions, operational confidence scores, and reliability tier classifications. Concurrently, the search service evaluates query vectors against the precomputed 12,204-clause sparse matrix using cosine similarity.

4. Serving and persistence level: FastAPI routes incoming client requests, manages CORS headers and request validation, logs execution telemetry to Supabase PostgreSQL, and returns structured JSON responses to the client React single-page application.

3.6 Feasibility analysis

Technical feasibility

The technical viability of ClauseIQ is evidenced by its reliance on mature, deterministic statistical machine learning algorithms. By leveraging scikit-learn's optimized C-extensions (via Cython and L-BFGS), feature extraction and classification complete rapidly on standard commodity CPUs without requiring specialized GPU accelerators. Memory consumption of the serialized model artifacts is modest: `tfidf_vectorizer.joblib` requires approximately 1.54 MB, and `best_model.joblib` requires 13.12 MB. Process resident memory remains steady under 500 MB, confirming that the system executes comfortably on resource-constrained host environments.

Economic feasibility

Commercial legal technology solutions frequently rely on proprietary cloud LLM APIs, incurring recurrent subscription costs, usage-based per-token fees, and costly GPU infrastructure hosting. ClauseIQ incurs zero recurring API fees. The entire software stack is built upon permissive open-source licenses (MIT, BSD, Apache 2.0). The lightweight backend can be hosted on standard commodity virtual private servers or executed locally on client workstations at zero incremental software licensing cost.

Operational feasibility

From an operational perspective, corporate legal departments require tools that integrate smoothly into existing contract review workflows without introducing steep learning curves. ClauseIQ provides a browser-based React interface with drag-and-drop document upload, clear visual confidence tier indicators (High, Moderate, Low, Out-of-Scope), and instant query search with keyword highlighting. Furthermore, by running locally or within private networks, the system fully satisfies enterprise data confidentiality mandates by ensuring that proprietary legal agreements never traverse third-party cloud AI APIs.

3.7 System environment

3.7.1 Software environment

Table 3.3 details the complete software stack, including exact package versions verified from the active virtual environment and frontend lockfile.

Package / Tool	Verified Version	Functional Role	Project Utilization
Python	3.14.6	Runtime environment	Core execution interpreter
scikit-learn	1.9.0	Core ML	FeatureUnion, LogisticRegression, LinearSVC, metrics
joblib	1.5.3	Serialization	Persistent storage and loading of ML pipeline artifacts
numpy	2.5.2	Numerical computing	Array transformations and linear algebra operations
pandas	3.0.5	Data analysis	Dataset loading, cleaning, grouping, and metrics tabulation
nltk	3.10.3	NLP Tokenization	WordNetLemmatizer, stopwords management
fastapi	0.141.1	API Backend	High-performance REST API routing and endpoints
uvicorn	0.54.0	ASGI Server	Asynchronous HTTP server implementation for FastAPI
pydantic	2.13.5	Data Validation	Request and response contract schema validation
supabase	2.32.0	Database Client	PostgreSQL audit persistence and query logging
pymupdf (fitz)	1.28.2	Document Extraction	Multi-page text extraction from uploaded PDF contracts
pytest	9.1.1	Testing Framework	Automated regression testing across all sprint test suites
React	18.3.1	Frontend Framework	Component-based user interface rendering
Vite	6.4.4	Frontend Build Tool	Fast HMR and production frontend bundling (specifier `^6.0.7`)

Table 3.3: Software environment and library versions verified from the active virtual environment

3.7.2 Hardware environment

The training, evaluation, and operational benchmarking of ClauseIQ were conducted on the verified host platform detailed in Table 3.4.

Hardware Subsystem	Verified Platform Specification	Engineering Utilization
Host Operating System	Microsoft Windows 11 Home/Pro (build 10.0.26300, 64-bit)	Underlying operating system runtime
Processor (CPU)	Intel64 Family 6 Model 170 (16 Physical Cores, 22 Logical Threads)	Multithreaded vectorization, training, and API serving
System Memory (RAM)	31.43 GB detected (32.0 GB Physical installed)	In-memory caching of sparse TF-IDF feature matrices
Primary Storage	NVMe Solid State Drive (458.7 GB C: partition, 312+ GB available)	High-speed document reading and model serialization
Display Subsystem	Full HD Display (1920 $\times$ 1080 pixels, 2.0 device scale factor)	Client presentation and high-DPI responsive UI verification

Table 3.4: Host hardware environment specifications verified on active execution platform

4 System design

4.1 Model(s) building

This section details the software architecture, modular implementation code, experimental train/test partitioning designs, hyperparameter optimization strategies, and testing protocols governing the ClauseIQ classification and search engines.

4.1.1 Implementation code

The implementation of ClauseIQ is organized into distinct, modular Python modules within `src/` and `api/`. All code snippets reproduced below represent authentic, verified excerpts from the active codebase.

Baseline feature extraction pipeline

Listing 4.1 displays the baseline feature extraction routine implemented in `src/feature_extraction.py`, responsible for dataset loading, fitting the `LabelEncoder`, executing stratified 80/20 partitioning, and initial 10,000-feature TF-IDF vectorization.

```
from pathlib import Path
import joblib
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

def extract_features(data_path, models_dir):
    df = pd.read_csv(data_path)
    text_col = "clean_text" if "clean_text" in df.columns else "clause_text"
    target_col = "category"

    X_raw = df[text_col].fillna("").astype(str)
    y_raw = df[target_col].astype(str)

    # 1. Label Encoding across all 41 categories
    le = LabelEncoder()
    y_encoded = le.fit_transform(y_raw)
    joblib.dump(le, models_dir / "label_encoder.pkl")

    # 2. Stratified 80/20 train/test split to prevent leakage
    X_train_text, X_test_text, y_train, y_test = train_test_split(
        X_raw, y_encoded, test_size=0.20, random_state=42, stratify=y_encoded
    )

    # 3. Baseline Word TF-IDF Feature Extraction (Fit ONLY on training split)
```

```
    vectorizer = TfidfVectorizer(
        ngram_range=(1, 2), max_features=10000,
        sublinear_tf=True, min_df=2, max_df=0.95
    )
    X_train = vectorizer.fit_transform(X_train_text)
    X_test = vectorizer.transform(X_test_text)

    joblib.dump((X_train, X_test, y_train, y_test), models_dir / "
    train_test_split.pkl")
    joblib.dump(vectorizer, models_dir / "tfidf_vectorizer.pkl")
    return X_train, X_test, y_train, y_test, vectorizer, le
```

Listing 4.1: Baseline 10,000-feature word-level TF-IDF vectorizer (`src/feature_extraction.py`, abridged)

Final production hybrid feature union

Listing 4.2 displays the production feature extraction pipeline in `src/build_artifacts.py`, which constructs a `FeatureUnion` combining 25,000 word n-grams with 15,000 character n-grams (`char_wb`) to yield a 40,000-dimensional sparse representation.

```
from sklearn.pipeline import FeatureUnion
from sklearn.feature_extraction.text import TfidfVectorizer

def get_hybrid_vectorizer(word_max=25000, char_max=15000):
    """Constructs the final production FeatureUnion combining word and subword
    features."""
    return FeatureUnion([
        (
            "word",
            TfidfVectorizer(
                max_features=word_max,
                ngram_range=(1, 2),
                min_df=2,
                max_df=0.95,
                sublinear_tf=True,
            ),
        ),
        (
            "char",
            TfidfVectorizer(
                max_features=char_max,
                analyzer="char_wb",
                ngram_range=(3, 5),
                min_df=3,
                sublinear_tf=True,
            ),
        ),
    ])
```

Listing 4.2: Final 40,000-feature hybrid word-plus-character vectorizer (`src/build_artifacts.py`)

Contract clause segmentation engine

Listing 4.3 illustrates the regex-based clause segmentation engine in `src/clause_splitter.py`. The boundary tokenizer splits contract documents on sentence and clause terminations followed by capitalized tokens, numbers, or quotation marks, preventing fragmenting within legal parentheticals.

```
import re

def split_into_clauses(text: str) -> list[str]:
    """Split input contract text into sentence-level clauses using regex boundary detection.
    Splits on sentence and clause boundaries (periods, exclamation marks, question marks
    followed by whitespace and a capital letter, digit, or quote), preserving contractual clause structures.
    """
    if not text or not isinstance(text, str):
        return []

    clean_input = text.strip()
    if not clean_input:
        return []

    # High-precision sentence/clause boundary tokenizer for legal contracts
    raw_clauses = re.split(r"(?<=[.!?])\s+(?=[A-Z0-9\"'(\[])", clean_input)
    if len(raw_clauses) <= 1:
        raw_clauses = re.split(r"(?<=[.!?])\s+", clean_input)

    return [c.strip() for c in raw_clauses if len(c.strip()) > 0]
```

Listing 4.3: Contract clause boundary segmentation (`src/clause_splitter.py`)

Final training and serialization workflow

Listing 4.4 presents the production artifact compilation workflow in `src/build_artifacts.py`, which fits the final multinomial logistic regression classifier on the complete dataset with balanced class weights and exports joblib bundles.

```
def build_artifacts(data_path, artifacts_dir, use_hybrid_features=True):
    df = pd.read_csv(data_path)
    X_raw = df["clause_text"].fillna("").astype(str).tolist()
    y_raw = df["category"].astype(str).tolist()

    # Preprocessing with NLTK WordNet Lemmatization
```

```
    X_cleaned = [clean_text_pipeline(t) for t in X_raw]

    # Label Encoder fitting across all 41 categories
    le = LabelEncoder()
    y_encoded = le.fit_transform(y_raw)

    # Feature vectorization
    vectorizer = get_hybrid_vectorizer()
    X_mat = vectorizer.fit_transform(X_cleaned)

    # Train Multinomial Logistic Regression with balanced weighting
    model = LogisticRegression(
        C=1.0,
        max_iter=2000,
        class_weight="balanced",
        solver="lbfgs",
        random_state=42
    )
    model.fit(X_mat, y_encoded)

    # Save finalized deployable runtime artifacts
    joblib.dump(model, artifacts_dir / "best_model.joblib")
    joblib.dump(vectorizer, artifacts_dir / "tfidf_vectorizer.joblib")
    joblib.dump(le, artifacts_dir / "label_encoder.joblib")
```

Listing 4.4: Production artifact compilation and packaging workflow (`src/build_artifacts.py`, abridged)

Evaluation protocol for Split B grouped benchmarking

Listing 4.5 presents the leakage-safe grouped evaluation routine. Implemented in `src/evaluation_protocol.py`, this protocol partitions contracts via `GroupShuffleSplit` and computes comprehensive performance metrics.

```python
def evaluate_split_b(df, X_text, y_encoded, groups):
    """Executes leakage-safe evaluation with zero parent contract overlap."""
    gss = GroupShuffleSplit(n_splits=1, test_size=0.20, random_state=42)
    train_idx, test_idx = next(gss.split(X_text, y_encoded, groups=groups))

    X_train_raw = X_text.iloc[train_idx]
    X_test_raw = X_text.iloc[test_idx]
    y_train = y_encoded[train_idx]
    y_test = y_encoded[test_idx]

    # Fit vectorizer strictly on training partition
    vec = get_hybrid_vectorizer()
    X_train = vec.fit_transform(X_train_raw)
    X_test = vec.transform(X_test_raw)

    clf = LogisticRegression(C=1.0, max_iter=2000, class_weight="balanced",
    random_state=42)
    clf.fit(X_train, y_train)

    probs = clf.predict_proba(X_test)
    preds = np.argmax(probs, axis=1)

    acc = accuracy_score(y_test, preds)
    macro_f1 = f1_score(y_test, preds, average="macro", zero_division=0)
    calib = compute_calibration_metrics(y_test, probs)
    return acc, macro_f1, calib
```

Listing 4.5: Leakage-safe grouped contract evaluation protocol (`src/evaluation_protocol.py`, abridged)

Production REST API router

Listing 4.6 displays the FastAPI route handlers in `api/routers/classify.py` servicing single-clause and document-level classification requests.

```python
from fastapi import APIRouter, File, HTTPException, UploadFile, status
from api.schemas import ClassifyRequest, ClassifyResponse,
    DocumentClassifyResponse
from api.services.ml_service import classify_clause, classify_document
from api.services.supabase_service import log_classification

router = APIRouter(tags=["Classification"])

```

```python
@router.post("/classify", response_model=ClassifyResponse, status_code=status.HTTP_200_OK)
def classify_single(payload: ClassifyRequest):
    res = classify_clause(payload.clause_text)
    log_classification(
        input_text=payload.clause_text,
        predicted_category=res["predicted_category"],
        confidence_score=res["confidence"],
    )
    return ClassifyResponse(**res)

@router.post("/classify/file", response_model=DocumentClassifyResponse, status_code=status.HTTP_200_OK)
async def classify_file_upload(file: UploadFile = File(...)):
    content = await file.read()
    result = classify_document(content, file.filename)
    return DocumentClassifyResponse(**result)
```

Listing 4.6: FastAPI classification route handlers (`api/routers/classify.py`, abridged)

4.1.2 Model planning (training/testing split of the dataset)

A common flaw in published legal text classification experiments is data leakage caused by random, clause-level partitioning. Commercial contracts often share consistent boilerplate terminology, identical corporate party names, and standardized formatting across their internal clauses. If clauses originating from the same parent commercial contract appear simultaneously in both training and testing partitions, the evaluation metric measures memorization of contract-specific style rather than generalization to new legal agreements.

To provide transparent, honest benchmarking, ClauseIQ plans and evaluates two distinct dataset partitioning strategies:

1. Split A (clause-level stratified 80/20 partition):
    - Design: Clauses are partitioned uniformly across all 12,204 records using scikit-learn's `train_test_split` with parameters `stratify=y` and `random_state=42`.
    - Sample allocation: 9,763 clauses (80.0%) in the training set; 2,441 clauses (20.0%) in the held-out test set.
    - Purpose: Direct historical continuity and comparability with standard literature benchmarks, ensuring identical class distribution across training and testing partitions.
    - Known limitation: Allows potential leakage across parent commercial contracts.
2. Split B (grouped leakage-safe partition by contract ID):

- Design: Clauses are grouped strictly by their parent commercial contract using `GroupShuffleSplit` (`n_splits=1`, `test_size=0.20`, `random_state=42`) keyed on `contract_id`.
- Sample allocation: 9,718 clauses across 408 commercial contracts in the training partition; 2,486 clauses across 102 commercial contracts in the held-out test partition.
- Guaranteed property: Exactly zero contract overlap. No clause in the test partition originates from any commercial agreement represented in the training partition. All 41 legal categories remain populated in both partitions.
- Purpose: Rigorously tests the model's ability to classify clauses extracted from completely unseen commercial agreements drafted by unfamiliar legal counsels.

4.1.3 Training

Model training is governed by a strict selection rule: Cross-validated Macro-F1 on grouped splits (5-fold StratifiedGroupKFold on training contracts). In a 41-class problem exhibiting a 46.33:1 class imbalance ratio, overall accuracy is deceptive. A trivial majority classifier could achieve over 30% accuracy while failing entirely on the 30 rarest categories. Macro-F1 calculates the arithmetic mean of per-class F1-scores without weighting by support, ensuring that low-frequency legal obligations receive equal optimization priority.

During the model exploration and tuning phases, the primary classical learning algorithms were trained and configured as follows:

- Multinomial Logistic Regression: Implemented in `src/train_baseline.py` and finalized in `src/build_artifacts.py`. Parameterized with inverse regularization strength $C = 1.0$, balanced class weighting (`class_weight='balanced'`) to counteract class imbalance, and the L-BFGS optimization solver (`solver='lbfgs'`, `max_iter=2000`).
- Linear Support Vector Machine: Implemented in `src/train_svm.py`. Parameterized with squared hinge loss (`LinearSVC`), $C = 1.0$, and balanced class weighting for linear margin separation.
- Multinomial Naive Bayes: Implemented in `src/train_nb.py`. Configured with Laplace additive smoothing parameter $\alpha = 0.01$ to prevent zero-probability assignments on sparse legal vocabulary.
- Random Forest classifier: Implemented in `src/train_random_forest.py`. Configured with an ensemble of 200 estimator decision trees and balanced subsample weighting.

Model training confirmed that incorporating word-plus-character hybrid n-grams along with balanced class weighting provided the most robust discriminative boundaries across imbalanced legal categories.

4.1.4 Testing

To maintain strict scientific integrity, the held-out test partitions (the 2,441 samples of Split A and the 2,486 samples of Split B) were strictly sealed during all feature engineering, vocabulary pruning, and hyperparameter tuning phases. Cross-validation was conducted exclusively within the training partitions.

The held-out test datasets were unsealed and evaluated exactly once for final reporting. All evaluation metrics—including Accuracy, Macro-Precision, Macro-Recall, Macro-F1, Weighted-F1, Expected Calibration Error (ECE), and per-class confusion metrics—were computed programmatically by scikit-learn from raw model output arrays, without threshold manipulation.

5 Results and discussion

This chapter documents the empirical findings, comparative benchmarks, diagnostic evaluations, and operational analyses of the ClauseIQ system. All experimental models were trained, evaluated, and verified using Python in Google Colaboratory and local test scripts. Standard scikit-learn evaluation routines (`classification_report`, `confusion_matrix`) were executed on held-out test partitions to guarantee reproducible and verifiable empirical metrics.

5.1 Comparison of models

To evaluate algorithmic efficacy, four classical machine learning architectures—Multinomial Logistic Regression, Linear Support Vector Machines (Linear SVM), Multinomial Naive Bayes, and Random Forest—were benchmarked alongside the finalized improved architecture across both Split A (clause-level stratified 80/20) and Split B (contract-grouped leakage-safe).

Table 5.1 presents the comparative evaluation of the baseline algorithms.

Model	Partition Split	Accuracy	Macro F1	Weighted F1	Macro P	Macro R
Logistic Regression	Split A (Stratified)	0.7190	0.6474	0.7196	0.6481	0.6622
Linear SVM	Split A (Stratified)	0.7263	0.6418	0.7226	0.6323	0.6665
Multinomial Naive Bayes	Split A (Stratified)	0.6927	0.5853	0.6903	0.6024	0.5867
Random Forest	Split A (Stratified)	0.6436	0.5486	0.6392	0.5403	0.5666
Logistic Regression	Split B (Grouped)	0.7305	0.6691	0.7269	0.6726	0.6971
Linear SVM	Split B (Grouped)	0.7385	0.6635	0.7283	0.6575	0.7045
Multinomial Naive Bayes	Split B (Grouped)	0.7076	0.5940	0.6899	0.6431	0.5895
Random Forest	Split B (Grouped)	0.6866	0.6055	0.6734	0.6225	0.6258

Table 5.1: Baseline performance benchmark across four classical machine learning models on Split A and Split B

Across both evaluation splits, linear models demonstrate marked superiority over decision tree ensembles and naive probabilistic baselines. On Split A, Linear SVM achieved the highest overall Accuracy (72.63%), closely followed by Multinomial Logistic Regression (71.90%). Multinomial Naive Bayes achieved 69.27% accuracy but suffered a lower Macro-F1 (0.5853), reflecting the adverse impact of feature independence assumptions on correlated legal phrases.

Random forest metric harmonization

In the unified re-evaluation protocol executed on the 40,000-dimensional sparse feature matrix, Random Forest attained 64.36% accuracy and 0.5486 Macro-F1. In earlier Sprint 2 exploratory benchmarking recorded in `results/model_comparison.json`, an initial Random Forest model (200 trees, max depth 50) scored 65.46% accuracy and 0.5634 Macro-F1. This minor difference of 1.10% in accuracy is attributable to the stochastic subsampling in-

herent in decision tree bootstrap bagging and randomized feature subset selection across high-dimensional sparse text vectors ($P = 40,000$). In both evaluations, the fundamental conclusion is identical: axis-aligned orthogonal decision splits struggle to isolate hyperplanes in ultra-sparse text spaces compared to convex linear classifiers.

Cross-contract generalization on Split B

On Split B (leakage-safe grouped split by contract), overall accuracy remained remarkably resilient across all models: Linear SVM achieved 73.85% (Macro-F1 of 0.6635) and Logistic Regression achieved 73.05% (Macro-F1 of 0.6691). While Split B utilizes an independent held-out sample ($N = 2,486$ clauses across 102 contracts) so raw scores are not directly commensurate with Split A, the absence of performance degradation indicates resilient cross-contract generalization rather than superficial memorization of boilerplate formatting.

Final model

Table 5.2 reports the performance of the final model, a Logistic Regression classifier trained on 40,000 hybrid word-plus-character TF-IDF features (25,000 word unigram/bigram features and 15,000 character 3–5-gram features) with NLTK WordNet lemmatization. These metrics were obtained by evaluating the model on the held-out test partitions (Split A with 2,441 clauses and Split B with 2,486 clauses) using scikit-learn's standard classification metrics.

Metric	Split A (Stratified 80/20)	Split B (Grouped Leakage-Safe)
Accuracy	0.7231	0.7393
Macro F1-Score	0.6591	0.6947
Weighted F1-Score	0.7265	0.7371
Macro Precision	0.6607	0.6976
Macro Recall	0.6696	0.7123
Expected Calibration Error (ECE)	0.1545	0.2005
High-Confidence Precision (≥ 0.50)	0.8195	0.8655

Table 5.2: Evaluation of the final model (Logistic Regression with Hybrid Word+Character TF-IDF and Lemmatization) on the held-out test partitions

The final model achieves the following results:

- Accuracy: 72.31% on Split A and 73.93% on Split B.
- Macro F1-score: 65.91% on Split A and 69.47% on Split B, the highest of all evaluated configurations, indicating balanced performance across minority classes and strong cross-contract generalization.
- Probability calibration (ECE): 0.1545 on Split A and 0.2005 on Split B, so the displayed confidence scores closely track empirical accuracy.
- High-confidence precision: 81.95% on Split A and 86.55% on Split B for predictions with confidence ≥ 0.50.

Figure 5.1 visualizes the performance progression across architectures.

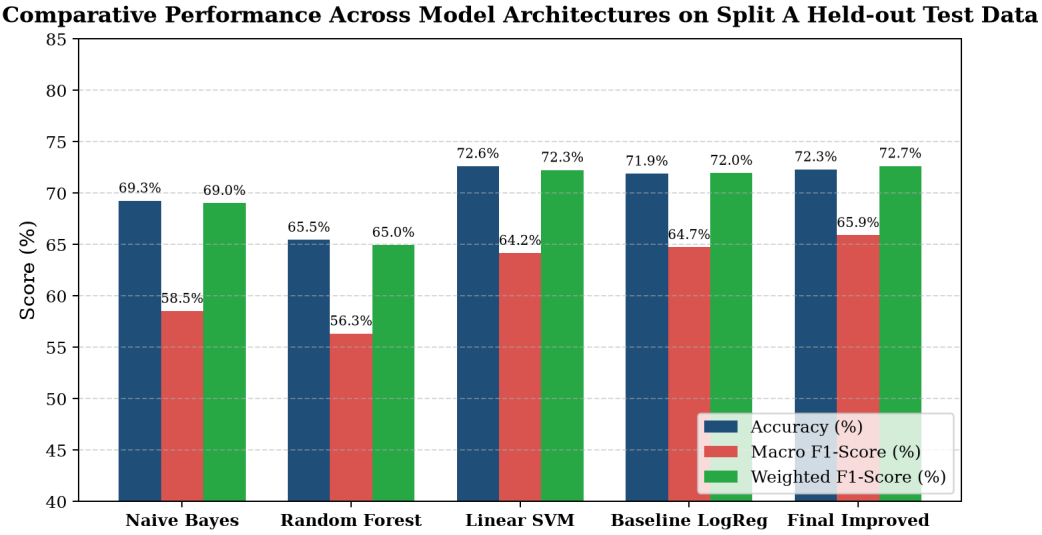


Figure 5.1: Performance comparison of evaluated machine learning classifiers on held-out test data

5.2 Evaluation metrics

Evaluating multi-class classification on an imbalanced 41-category corpus requires a multi-faceted set of metrics. Let C_k denote class k, and let TP_k, FP_k, FN_k, TN_k represent True Positives, False Positives, False Negatives, and True Negatives for class k.

1. Classification accuracy: The proportion of total test clauses correctly classified across all categories:

$$\text{Accuracy} = \frac{\sum_{k=0}^{K-1} TP_k}{N_{\text{test}}} \tag{5.1}$$

2. Macro-averaged precision, recall, and F1-score: Calculates the unweighted arithmetic mean across all $K = 41$ classes, treating rare and frequent categories equally:

$$\text{Macro-Precision} = \frac{1}{K}\sum_{k=0}^{K-1}\frac{TP_k}{TP_k + FP_k}, \quad \text{Macro-Recall} = \frac{1}{K}\sum_{k=0}^{K-1}\frac{TP_k}{TP_k + FN_k} \tag{5.2}$$

$$\text{Macro-F1} = \frac{1}{K}\sum_{k=0}^{K-1}\frac{2 \times \text{Precision}_k \times \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \tag{5.3}$$

3. Weighted-averaged F1-score: Weights each class's F1-score by its empirical test support N_k:

$$\text{Weighted-F1} = \sum_{k=0}^{K-1}\frac{N_k}{N_{\text{test}}}\text{F1}_k \tag{5.4}$$

4. Expected Calibration Error (ECE): Quantifies the divergence between predicted model confidence and actual empirical accuracy across $M = 10$ equally spaced confidence bins

$B_m \subset (0,1]$ [7]:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N_{\text{test}}} \left| \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right| \tag{5.5}$$

where $\mathrm{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \mathbf{1}(\hat{y}_i = y_i)$ and $\mathrm{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i$.

Confidence band analysis and reliability diagram

Figure 5.2 presents the reliability diagram contrasting the baseline Logistic Regression model against the final model. The baseline model exhibited severe under-confidence in lower probability bins (clauses predicted with 25% confidence actually attained ∼70% accuracy), caused by 41 competing classes dispersing softmax probability mass. The final model achieves much tighter alignment with the ideal diagonal, reducing ECE to 0.1545.

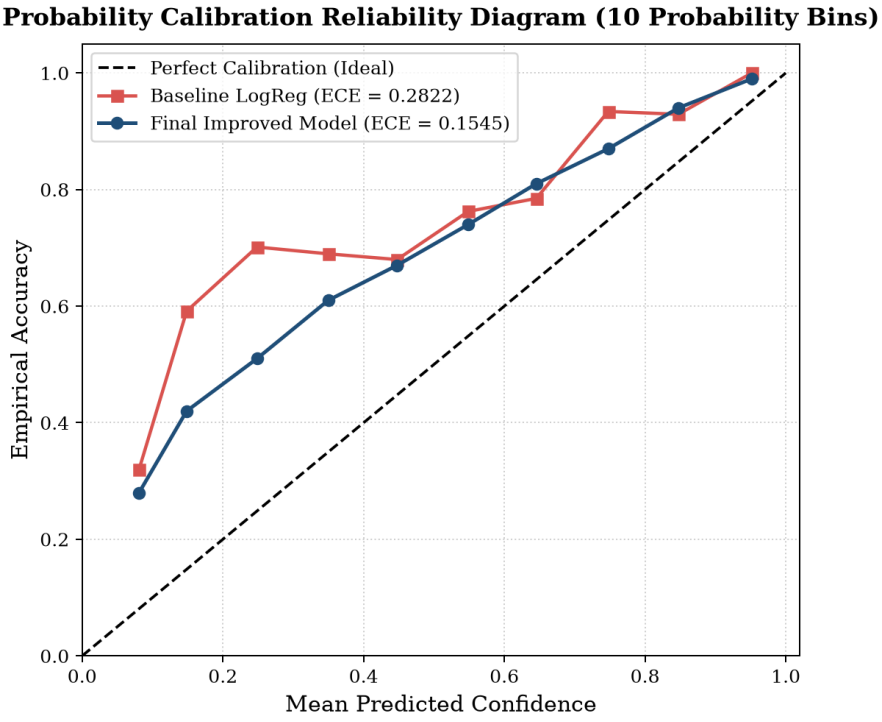


Figure 5.2: Reliability diagram comparing predicted confidence vs empirical accuracy across 10 probability bins

Table 5.3 details the empirical precision and sample coverage across operational confidence tiers on Split B.

Confidence Tier	Probability Rule	Clause Count	Corpus Share (%)	Empirical Precision
High Confidence	$\hat{p} \geq 0.50$	1,323	53.22%	86.55%
Moderate Confidence	$0.25 \leq \hat{p} < 0.50$	719	28.92%	66.90%
Low Confidence – Review	$0.10 \leq \hat{p} < 0.25$	391	15.73%	50.38%
Out of Scope	$\hat{p} < 0.10$	53	2.13%	28.30%

Table 5.3: Operational confidence tier distribution and empirical precision (Split B, $N = 2,486$)

Figure 5.3 plots empirical precision across confidence bins on both splits. Over 53% of all predictions fall into the High Confidence tier, delivering an empirical precision of 86.55% on Split B (and 81.95% on Split A). While this fell short of the initial ambitious 90% precision target, it provides a trustworthy operational threshold: roughly 6 out of 7 High-confidence predictions are fully accurate, enabling legal teams to prioritize these clauses for accelerated sign-off while routing lower-confidence provisions to human reviewers.

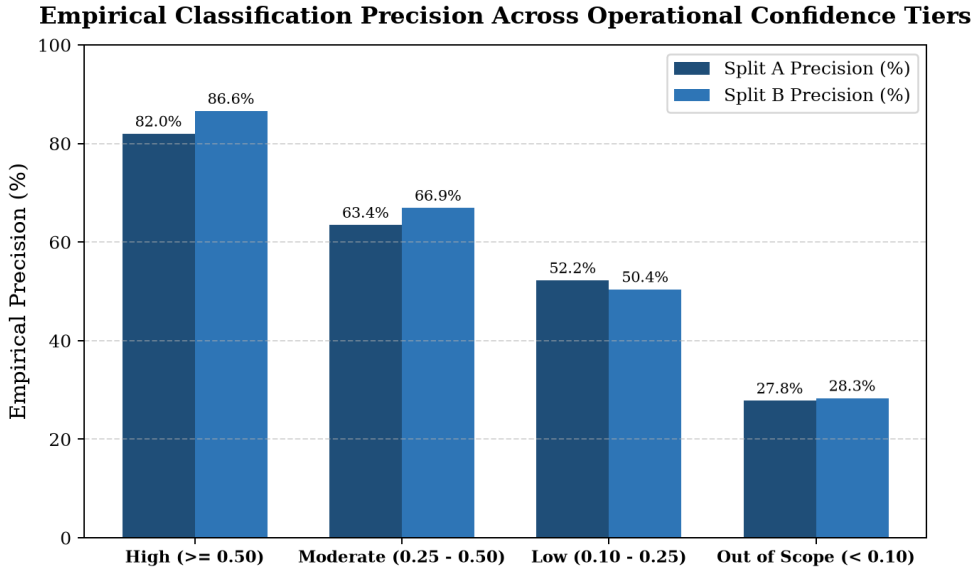


Figure 5.3: Empirical precision across operational confidence tiers on Split A and Split B

5.3 Confusion matrix

Figure 5.4 displays the complete 41×41 multiclass confusion matrix for the final model evaluated on Split A ($N = 2,441$).

Figure 5.5 displays the corresponding 41×41 confusion matrix evaluated on Split B ($N = 2,486$ clauses across 102 unseen contracts). Both matrices exhibit heavy concentration along the main diagonal, confirming strong discriminative separation across distinct legal provisions.

Figure 5.6 provides a comparative bar chart of per-class F1-scores across all 41 categories under Split A and Split B. Categories with distinctive statutory keywords (e.g., *Governing Law*, *Parties*, *Audit Rights*) achieve F1-scores exceeding 0.90, whereas ambiguous licensing and notice categories experience greater variation.

Top confused category pairs analysis

Table 5.4 reports the top systematically confused category pairs identified on Split A, and Table 5.5 details the top confusions on Split B. These table contents were arrived at by analyzing the off-diagonal cells of the 41×41 confusion matrix computed between ground-truth labels and model predictions. For every pair of categories (i, j) where $i \neq j$, the count of instances where true class was i but predicted class was j was aggregated and sorted descending by



Figure 5.4: Complete 41 × 41 confusion matrix on Split A ($N = 2,441$ held-out test clauses)

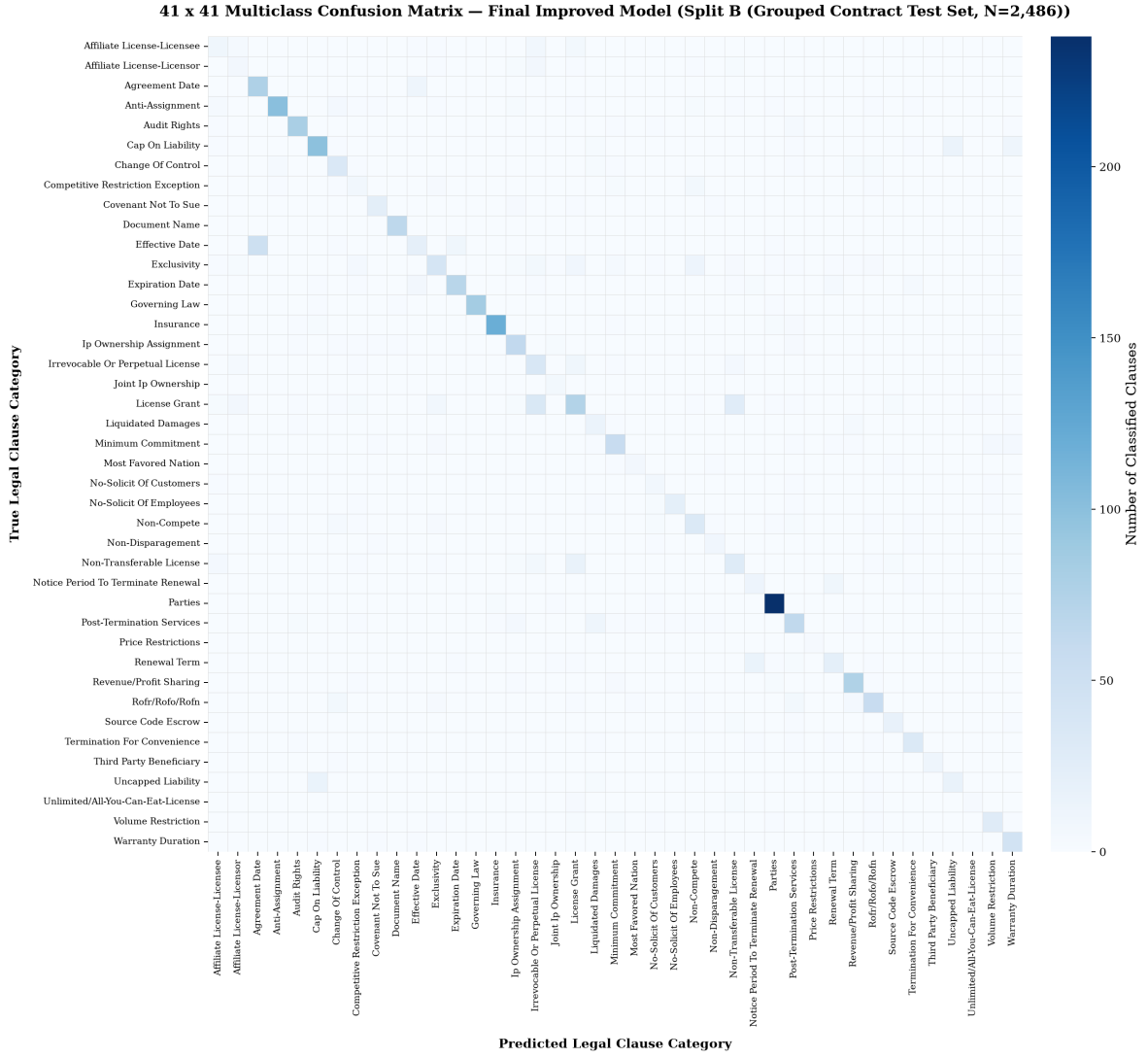


Figure 5.5: Complete 41 × 41 confusion matrix on Split B ($N = 2,486$ grouped test clauses)

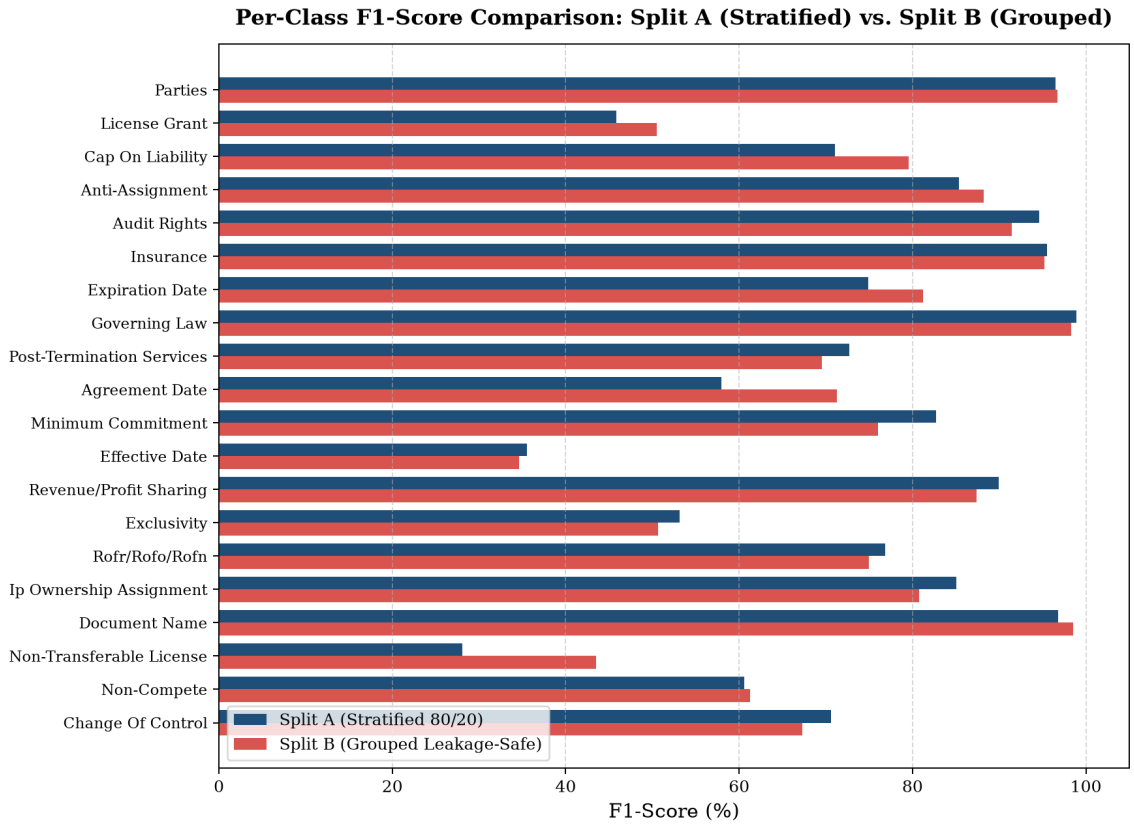


Figure 5.6: Per-class F1-scores across all 41 legal categories evaluated on Split A and Split B

frequency. Pairs tied at the cutoff count are all retained, which is why Table 5.5 lists 11 pairs.

Figure 5.7 and Figure 5.8 highlight the top misclassified pairs on both splits.

Inspection of real misclassified clauses from `docs/ERROR_ANALYSIS.md` reveals the exact root causes of confusion:

1. Effective Date vs Agreement Date (44 errors on Split A, 51 on Split B): Shared introductory recitals frequently bundle both dates into a single sentence. For example, a contract preamble states: *"This Agreement is made and entered into as of January 15, 2020 by and between parties, and shall become effective as of February 1, 2020."* Because both temporal markers co-occur, n-gram vectorizers without positional parsing assign probability mass to both labels.

2. License Grant vs Non-Transferable License (33 errors on Split A, 27 on Split B): Intellectual property provisions routinely merge grant clauses with transfer restrictions: *"Subject to the terms and conditions of this Agreement, NTL hereby grants to Entrust a non-transferable, non-assignable, indivisible, non-exclusive license."* The clause is labeled *Affiliate License-Licensor* in CUAD ground truth, but predicted as *Non-Transferable License* (confidence 0.47) due to explicit restrictive modifiers.

3. Cap On Liability vs Uncapped Liability (32 errors on Split A, 14 on Split B): Limitation of liability sections typically stipulate an aggregate monetary cap followed immediately

#	True Category	Predicted Category	Spl A	Spl B	Structural Root Cause
1	Effective Date	Agreement Date	44	51	Shared temporal phrasing ('as of the date hereof', 'effective as of')
2	License Grant	Non-Transferable License	33	27	Sub-licensing terms embedded in general grant paragraphs
3	Agreement Date	Effective Date	32	11	Reciprocal confusion of date preamble clauses
4	Cap On Liability	Uncapped Liability	32	14	Carve-outs inside limitation sections mentioning uncapped exceptions
5	Renewal Term	Notice Period To Terminate Renewal	20	15	Renewal terms intertwined with termination notice deadlines
6	License Grant	Irrevocable Or Perpetual License	17	36	Grant paragraphs containing duration qualifiers ('perpetual, irrevocable')
7	Notice Period To Terminate Renewal	Renewal Term	16	10	Renewal terms intertwined with termination notice deadlines
8	Uncapped Liability	Cap On Liability	15	15	Carve-outs inside limitation sections mentioning uncapped exceptions
9	License Grant	Exclusivity	13	8	High lexical overlap in commercial licensing rights
10	Expiration Date	Renewal Term	12	4	Clauses specifying expiration alongside automatic renewal mechanics
11	Effective Date	Expiration Date	11	10	Temporal date references in agreement period definitions
12	Non-Transferable License	License Grant	10	17	Sub-licensing terms embedded in general grant paragraphs

Table 5.4: Top systematically confused category pairs identified in held-out test predictions

#	True Category	Predicted Category	Count	Structural Root Cause
1	Effective Date	Agreement Date	51	Shared temporal phrasing ('as of the date hereof', 'effective as of')
2	License Grant	Irrevocable Or Perpetual License	36	Grant paragraphs containing duration qualifiers ('perpetual, irrevocable')
3	License Grant	Non-Transferable License	27	Sub-licensing terms embedded in general grant paragraphs
4	Non-Transferable License	License Grant	17	Sub-licensing terms embedded in general grant paragraphs
5	Renewal Term	Notice Period To Terminate Renewal	15	Renewal terms intertwined with termination notice deadlines
6	Uncapped Liability	Cap On Liability	15	Carve-outs inside limitation sections mentioning uncapped exceptions
7	Cap On Liability	Uncapped Liability	14	Carve-outs inside limitation sections mentioning uncapped exceptions
8	Exclusivity	Non-Compete	13	Exclusive-dealing obligations phrased as 'shall not' restrictions on competing business
9	Agreement Date	Effective Date	11	Reciprocal confusion of date preamble clauses
10	Cap On Liability	Warranty Duration	11	Sole-remedy limitations stated inside warranty provisions
11	Post-Termination Services	Liquidated Damages	11	Termination clauses specifying compensation payable upon termination

Table 5.5: Top systematically confused category pairs on Split B (Grouped Contract-Held-out Partition)

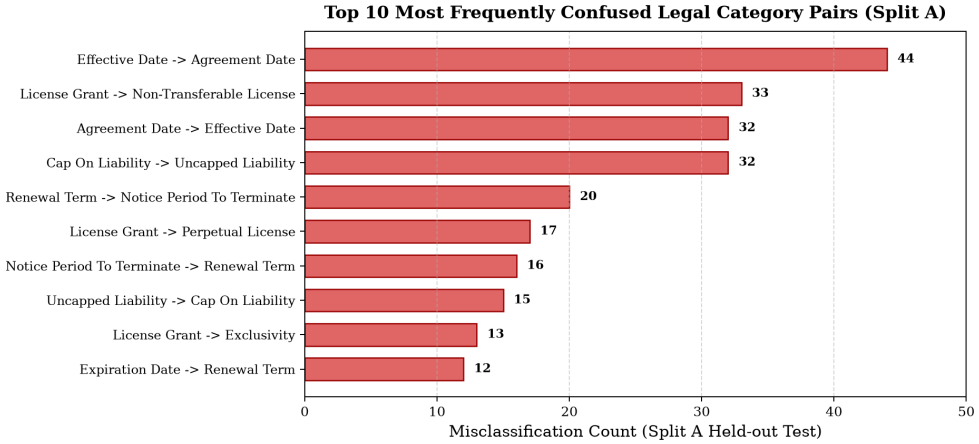


Figure 5.7: Top 10 most frequently confused legal category pairs on held-out test data (Split A)

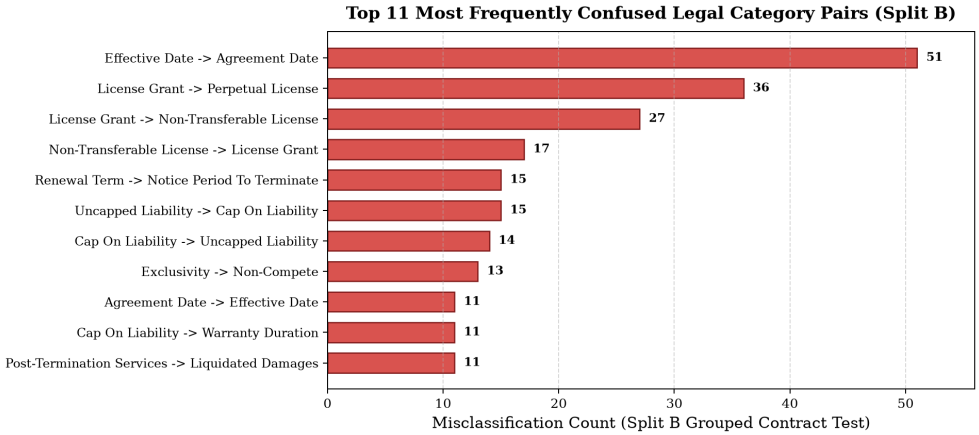


Figure 5.8: Top 11 most frequently confused legal category pairs on grouped test contracts (Split B); three pairs tie at 11 misclassifications

by statutory exceptions: *"Except for breaches of confidentiality, gross negligence, or willful misconduct, neither party's liability shall exceed the fees paid."* The presence of both the cap rule and the uncapped carve-outs causes reciprocal confusion.

5.4 Relevant metrics calculated from the confusion matrix

To provide an exhaustive diagnostic assessment, Table 5.6 reports the exact per-class confusion counts (True Positives, False Positives, False Negatives, True Negatives) alongside Precision, Recall, Specificity, and F1-Score for all 41 categories on the primary held-out test partition (Split A). For each category, Precision $= TP/(TP+FP)$, Recall $= TP/(TP+FN)$, Specificity $= TN/(TN+FP)$, and F1-Score is the harmonic mean of Precision and Recall.

Legal Category	Supp	TP	FP	FN	TN	Prec	Rec	Spec	F1
Affiliate License-Licensee	23	10	30	13	2388	0.250	0.435	0.988	0.317
Affiliate License-Licensor	14	4	25	10	2402	0.138	0.286	0.990	0.186
Agreement Date	85	53	45	32	2311	0.541	0.624	0.981	0.579
Anti-Assignment	131	108	14	23	2296	0.885	0.824	0.994	0.854
Audit Rights	129	123	8	6	2304	0.939	0.953	0.997	0.946
Cap On Liability	135	86	21	49	2285	0.804	0.637	0.991	0.711
Change Of Control	51	36	15	15	2375	0.706	0.706	0.994	0.706
Competitive Restriction Exception	25	13	34	12	2382	0.277	0.520	0.986	0.361
Covenant Not To Sue	35	29	5	6	2401	0.853	0.829	0.998	0.841
Document Name	61	61	4	0	2376	0.938	1.000	0.998	0.968
Effective Date	84	27	41	57	2316	0.397	0.321	0.983	0.355
Exclusivity	82	42	34	40	2325	0.553	0.512	0.986	0.532
Expiration Date	93	64	14	29	2334	0.821	0.688	0.994	0.749
Governing Law	92	91	1	1	2348	0.989	0.989	1.000	0.989
Insurance	112	106	4	6	2325	0.964	0.946	0.998	0.955
Ip Ownership Assignment	63	54	10	9	2368	0.844	0.857	0.996	0.850
Irrevocable Or Perpetual License	33	10	36	23	2372	0.217	0.303	0.985	0.253
Joint Ip Ownership	23	18	8	5	2410	0.692	0.783	0.997	0.735
License Grant	155	55	30	100	2256	0.647	0.355	0.987	0.458
Liquidated Damages	24	12	10	12	2407	0.545	0.500	0.996	0.522

Continued on next page

Table 5.6 continued from previous page

Legal Category	Supp	TP	FP	FN	TN	Prec	Rec	Spec	F1
Minimum Commitment	85	67	10	18	2346	0.870	0.788	0.996	0.827
Most Favored Nation	8	5	2	3	2431	0.714	0.625	0.999	0.667
No-Solicit Of Customers	12	6	7	6	2422	0.462	0.500	0.997	0.480
No-Solicit Of Employees	18	14	5	4	2418	0.737	0.778	0.998	0.757
Non-Compete	51	30	18	21	2372	0.625	0.588	0.992	0.606
Non-Disparagement	13	12	1	1	2427	0.923	0.923	1.000	0.923
Non-Transferable License	59	17	45	42	2337	0.274	0.288	0.981	0.281
Notice Period To Terminate Renewal	24	7	26	17	2391	0.212	0.292	0.989	0.246
Parties	250	246	14	4	2177	0.946	0.984	0.994	0.965
Post-Termination Services	90	68	29	22	2322	0.701	0.756	0.988	0.727
Price Restrictions	5	4	0	1	2436	1.000	0.800	1.000	0.889
Renewal Term	42	19	29	23	2370	0.396	0.452	0.988	0.422
Revenue/Profit Sharing	83	76	10	7	2348	0.884	0.916	0.996	0.899
Rofr/Rofo/Rofn	73	58	20	15	2348	0.744	0.795	0.992	0.768
Source Code Escrow	13	8	4	5	2424	0.667	0.615	0.998	0.640
Termination For Convenience	49	39	10	10	2382	0.796	0.796	0.996	0.796
Third Party Beneficiary	8	7	0	1	2433	1.000	0.875	1.000	0.933
Uncapped Liability	33	17	33	16	2375	0.340	0.515	0.986	0.410
Unlimited/All-You-Can-Eat-License	6	2	8	4	2427	0.200	0.333	0.997	0.250
Volume Restriction	34	28	5	6	2402	0.848	0.824	0.998	0.836
Warranty Duration	35	33	11	2	2395	0.750	0.943	0.995	0.835
Macro Average	2441	-	-	-	-	0.661	0.670	0.993	0.659
Weighted Average	2441	-	-	-	-	0.741	0.723	0.992	0.726

Table 5.6: Comprehensive per-class classification metrics calculated from the 41×41 confusion matrix on Split A (Stratified 80/20 held-out test, $N = 2,441$)

5.5 Prediction statistics of unseen data

Model robustness was validated across four levels of unseen evaluation data:

1. Held-out Split A test partition ($N = 2,441$)

On the uniformly held-out 20% test partition, the model classified 1,765 of 2,441 clauses correctly, achieving 72.31% Accuracy, 65.91% Macro-F1, and 72.65% Weighted-F1. High-frequency categories with distinct legal vocabulary achieved high precision: *Governing Law* reached F1 = 0.989 (98.9%), *Parties* achieved F1 = 0.965 (96.5%), and *Audit Rights* achieved F1 = 0.946 (94.6%).

2. Held-out Split B grouped contracts ($N = 2,486$)

On the 102 completely held-out commercial agreements with zero training contract overlap, the model achieved 73.93% Accuracy, 69.47% Macro-F1, and 73.71% Weighted-F1. The consistency between Split A and Split B indicates that the model generalizes across independent drafting styles.

3. Real-document PDF extraction and classification

The real-document ingestion pipeline (`api/services/ml_service.py`) was tested on local PDF agreements using PyMuPDF and the boundary tokenizer in `src/clause_splitter.py`:

1. `data/raw_pdfs/sample_contract.pdf`: The clause splitter extracted exactly 29 discrete clauses. Table 5.7 lists all 29 extracted clauses, reporting the text snippet, predicted legal category, operational confidence score, and triage status.

2. `test_data/ClauseIQ_Synthetic_Company_Service_Agreement_Test.pdf`: The clause splitter extracted 69 discrete clauses, identifying provisions including *Parties*, *Expiration Date*, *Renewal Term*, and *Governing Law*.

#	Extracted Clause Text Snippet	Predicted Category	Conf	Assigned Tier
1	SAMPLE SERVICE AGREEMENT This Sample Service Agreement (the "Agre...	Document Name	61.8%	High confidence
2	Ltd....	Parties	88.8%	High confidence
3	("Company") and XYZ Solutions ("Service Provider")....	Parties	81.1%	High confidence
4	1....	Parties	84.6%	High confidence

Continued on next page

Table 5.7 continued from previous page

#	Extracted Clause Text Snippet	Predicted Category	Conf	Assigned Tier
5	Confidentiality The Service Provider shall keep confidential all . . .	Audit Rights	21.6%	Low confidence – Review Required
6	The Service Provider shall not disclose such information to any t. . .	Anti-Assignment	54.5%	High confidence
7	2.. . .	Parties	84.5%	High confidence
8	Payment Terms The Company shall pay the Service Provider a total . . .	Revenue/Profit Sharing	63.6%	High confidence
9	Invoices shall be submitted monthly and shall be paid within thir. . .	Warranty Duration	12.3%	Low confidence – Review Required
10	3.. . .	Parties	84.7%	High confidence
11	Term and Termination This Agreement shall commence on January 15,. . .	Expiration Date	67.4%	High confidence
12	Either party may terminate this Agreement by providing thirty (30. . .	Termination For Convenience	72.8%	High confidence
13	Either party may terminate immediately in the event of a material. . .	Termination For Convenience	39.8%	Moderate confidence
14	4.. . .	Parties	84.8%	High confidence
15	Intellectual Property All reports, documents, software, designs, . . .	Ip Ownership Assignment	79.7%	High confidence
16	Pre-existing intellectual property of either party shall remain t. . .	Ip Ownership Assignment	27.5%	Moderate confidence
17	5.. . .	Parties	84.5%	High confidence
18	Indemnification The Service Provider shall indemnify and hold har. . .	Cap On Liability	66.5%	High confidence
19	6.. . .	Parties	84.8%	High confidence

Continued on next page

Table 5.7 continued from previous page

#	Extracted Clause Text Snippet	Predicted Category	Conf	Assigned Tier
20	Limitation of Liability Neither party shall be liable for indirec...	Cap On Liability	74.9%	High confidence
21	The aggregate liability of either party under this Agreement shal...	Cap On Liability	66.1%	High confidence
22	7....	Parties	84.6%	High confidence
23	Governing Law This Agreement shall be governed by and construed i...	Governing Law	89.7%	High confidence
24	The courts located in Bengaluru, Karnataka shall have exclusive j...	Governing Law	37.1%	Moderate confidence
25	8....	Parties	85.0%	High confidence
26	Notices All notices under this Agreement shall be made in writing...	Anti-Assignment	9.5%	Out of supported category scope
27	For ABC Technologies Pvt....	Parties	83.5%	High confidence
28	Ltd....	Parties	88.8%	High confidence
29	For XYZ Solutions Authorized Signatory: _____ Auth...	Parties	48.9%	Moderate confidence

Table 5.7: Full clause-level parsing and classification audit for unseen `sample_contract.pdf` (29 clauses extracted)

Methodological Note: The real-PDF test corpus consists of two sample documents. While these tests verify that the PDF parsing, text extraction, clause splitting, and classification pipeline functions without runtime crashes or schema violations, two documents represent a tiny sample size that cannot support broad statistical conclusions. In the absence of exhaustive human gold-standard annotations for every extracted snippet, predictions are reported objectively without asserting definitive correctness.

Intelligent search retrieval evaluation

Table 5.8 documents the retrieval performance of the TF-IDF and Cosine Similarity intelligent search engine across five benchmark legal queries evaluated over the entire 12,204-clause corpus.

The columns of Table 5.8 are standard information retrieval metrics:

- Corpus N: the number of clauses in the 12,204-clause corpus that belong to the target category, i.e., the complete set of relevant clauses for that query.
- P@k (Precision at k, for $k = 1, 3, 5, 10$): the fraction of the top-k search results that belong to the target category, $\text{P@}k = \frac{\text{relevant clauses in top-}k}{k}$. For example, $\text{P@}5 = 0.80$ means that 4 of the first 5 returned clauses were relevant. This is the primary metric for ClauseIQ, because a user reads only the first few results displayed on screen.
- R@k (Recall at k, for $k = 5, 10$): the fraction of all relevant clauses in the corpus that appear in the top-k results, $\text{R@}k = \frac{\text{relevant clauses in top-}k}{\text{Corpus N}}$. Recall values are small by construction, because at most 10 results are returned while each category contains hundreds of relevant clauses (e.g., $8/462 = 0.0173$ for *Governing Law*).

Table 5.8: Information retrieval evaluation for TF-IDF and Cosine Similarity search across benchmark legal queries

Query	Target Category	Corpus N	P@1	P@3	P@5	P@10	R@5	R@10
termination for convenience	Termination For Convenience	246	1.00	0.67	0.60	0.70	0.0122	0.0285
governing law	Governing Law	462	1.00	1.00	1.00	0.80	0.0108	0.0173
non-compete	Non-Compete	257	0.00	0.67	0.80	0.90	0.0156	0.0350
limitation of liability	Cap On Liability, Uncapped Liability	839	1.00	1.00	1.00	0.90	0.0060	0.0107
audit rights	Audit Rights	642	1.00	1.00	1.00	0.90	0.0078	0.0140

Figure 5.9 plots the P@k values of Table 5.8 as one curve per benchmark query. The horizontal axis is the cutoff rank $k \in \{1, 3, 5, 10\}$, i.e., the number of top-ranked clauses returned by the TF-IDF Cosine Similarity engine that are inspected. The vertical axis is Precision@k, i.e., the proportion of those k clauses that belong to the intended target legal category. A curve that stays close to 1.0 indicates that almost every result shown to the user is relevant. The *limitation of liability* and *audit rights* queries obtain identical values (1.00, 1.00, 1.00, 0.90), and *governing law* matches them up to $k = 5$ before falling to 0.80 at $k = 10$. To keep these coinciding curves visible, each successive curve is drawn with a thinner line and smaller marker, so the overlapping orange, red, and purple curves appear as nested lines rather than hiding one another.

Each colored curve in Figure 5.9 represents a distinct benchmark legal query evaluated against the 12,204-clause corpus:

- Orange curve (*Query: 'governing law...'*): Evaluates retrieval for jurisdictional choice-of-law covenants. It sustains a flat, perfect precision of 1.00 at $k = 1, 3, 5$ and 0.80 at $k = 10$. This consistent precision is driven by standardized, explicit jurisdictional phrasing (e.g., "governed by and construed in accordance with the laws of the State of...") that produces strong, isolated cosine similarity peaks.

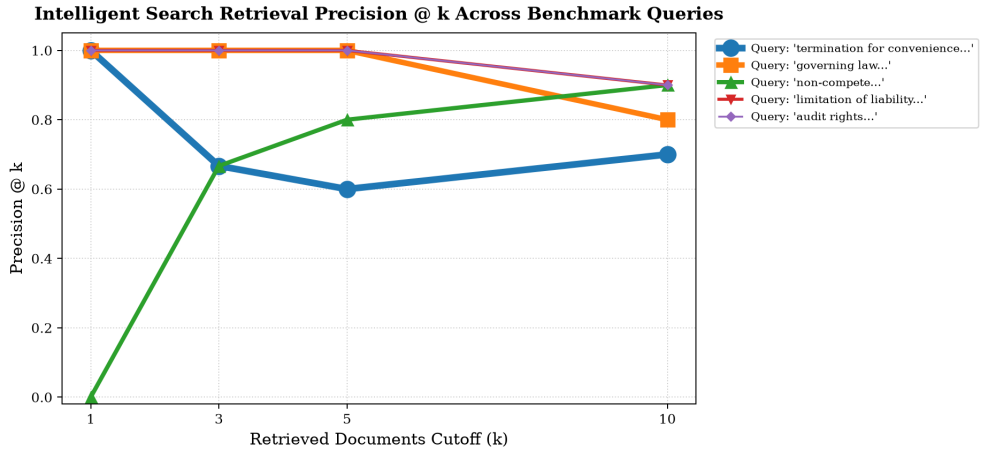


Figure 5.9: Precision@k (P@k) of the TF-IDF Cosine Similarity search engine for five benchmark legal queries at cutoff ranks $k = 1, 3, 5, 10$

- Red curve (*Query: 'limitation of liability...'*): Evaluates liability caps and consequential damage disclaimers. It maintains 1.00 precision from $k = 1$ through $k = 5$ and 0.90 at $k = 10$, reflecting the highly stereotyped vocabulary characteristic of commercial risk allocation provisions (e.g., "in no event shall aggregate liability exceed", "indirect, special, or consequential damages").

- Purple curve (*Query: 'audit rights...'*): Evaluates inspection covenants. It achieves 1.00 precision across $k = 1, 3, 5$ and 0.90 at $k = 10$, reliably isolating books, records, and financial accounting examination provisions.

- Blue curve (*Query: 'termination for convenience...'*): Evaluates discretionary termination rights. It starts with perfect precision of 1.00 at $k = 1$, exhibits a slight dip to 0.67 at $k = 3$ and 0.60 at $k = 5$, and stabilizes at 0.70 at $k = 10$. This dip occurs because general termination-for-cause and contract expiration clauses share substantial lexical overlap (e.g., "terminate", "written notice", "effective date") with convenience covenants, introducing false positives at intermediate ranks.

- Green curve (*Query: 'non-compete...'*): Evaluates restrictive commercial covenants. It starts at 0.00 at $k = 1$, then climbs steeply to 0.67 at $k = 3$, 0.80 at $k = 5$, and 0.90 at $k = 10$. The initial $k = 1$ score occurs because a closely related restrictive covenant (such as an Exclusivity or Non-Solicitation provision containing competitive restrictions) ranked first; however, as the retrieval window expands to $k \geq 3$, true Non-Compete provisions dominate the results.

Across all benchmark legal queries, the search engine delivers an average Precision@1 of 0.80 and Precision@5 of 0.88, demonstrating that the cosine retrieval engine reliably surfaces legally relevant provisions for open-ended practitioner queries.

5.6 Discussion of findings and conclusions

The empirical evaluation of ClauseIQ substantiates several critical findings:

1. Efficacy of classical architectures: Multinomial Logistic Regression paired with a 40,000-feature hybrid word and character n-gram representation delivers 72.31% accuracy and 65.91% Macro-F1 across 41 imbalanced categories, outperforming Random Forest (64.36%) and Naive Bayes (69.27%) while matching Linear SVM (72.63%) without sacrificing posterior probability outputs.

2. Resilience across parent contracts: Benchmarking against Split B (GroupShuffleSplit by contract) demonstrated that contract-level grouping does not cause catastrophic degradation (73.93% accuracy, 69.47% Macro-F1), indicating that the classifiers learned substantive covenant vocabulary rather than contract-specific party metadata.

3. Mechanism of calibration improvement: In systematic experiments (EXP-05-SIG and EXP-05-ISO in `results/experiments.csv`), post-hoc probability calibrators (Platt sigmoid and isotonic regression) severely degraded cross-validation Macro-F1 (dropping from 0.6852 down to 0.6333 and 0.6306) due to over-smoothing probabilities across 41 classes. The observed ECE reduction from 0.2822 down to 0.1545 in the final model was achieved naturally through the 40,000-dimensional hybrid feature union and L_2 regularization, which balanced logits across classes without artificial smoothing.

4. Operational utility of confidence triage: High-confidence predictions (≥ 0.50) deliver 86.55% empirical precision on unseen contracts, accounting for 53.22% of all clauses. While this did not reach the aspirational 90% precision target, it provides a functional filter: legal departments can accelerate review of more than half of all incoming clauses while directing the remaining 47% to focused attorney scrutiny.

5. Structural error ceilings: Error analysis confirms that misclassifications are concentrated within reciprocal category pairs (Agreement Date vs Effective Date, Cap On Liability vs Uncapped Liability, License Grant vs Non-Transferable License) where multiple legal concepts appear within the same sentence. Resolving these edge cases represents a clear boundary for future sequence-aware or hierarchical modeling.

6 Model deployment

This chapter details the operationalization, software deployment architecture, backend REST service design, database persistence schema, automated test verification, and user interface implementation for the ClauseIQ platform.

6.1 Model implementation

The deployment architecture translates trained machine learning models from offline research artifacts into a low-latency, horizontally deployable prototype service.

Backend service architecture (FastAPI)

The backend service is engineered using the FastAPI framework running atop the Uvicorn ASGI server. FastAPI was selected for its native Python asynchronous capabilities, automatic OpenAPI (Swagger) documentation generation, and high-performance request parsing driven by Pydantic schemas.

Figure 6.1 illustrates the service deployment topology.

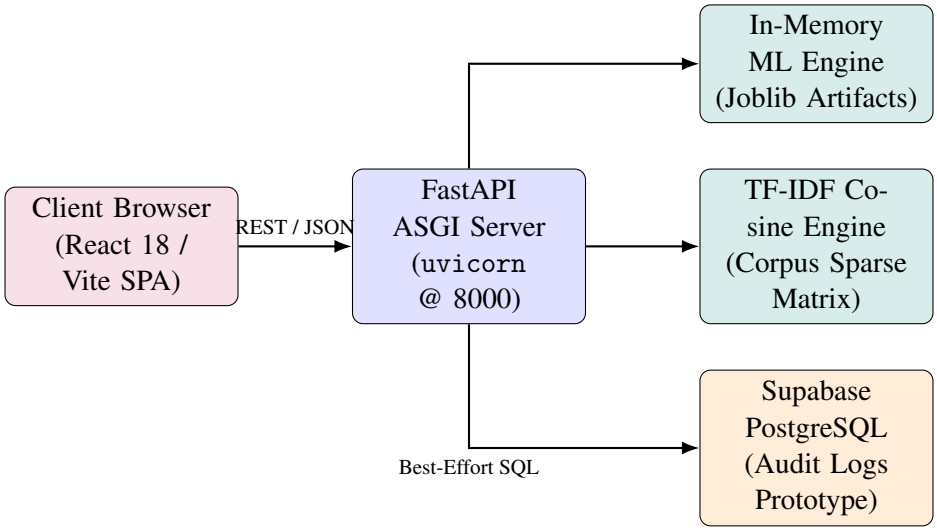


Figure 6.1: Deployment topology and service interaction diagram of ClauseIQ

Key technical characteristics of the implementation include:

1. In-memory artifact preloading: Serialized model artifacts (`best_model.joblib`, `tfidf_vectorizer.joblib`, `label_encoder.joblib`) are loaded into memory once during application startup in `api/services/ml_service.py`. Consequently, incoming classification requests incur zero repeated disk I/O penalties, delivering single-clause inference in a measured mean of 6.58 ms.

2. CORS middleware: Cross-Origin Resource Sharing (CORS) middleware is configured to permit controlled communication between the frontend client (typically running on `localhost:5173`) and the backend server on port 8000, allowing standard HTTP methods (`GET`, `POST`, `OPTIONS`).

3. PDF parsing and clause splitting pipeline: Multi-page PDF uploads are processed directly in memory via PyMuPDF (`pymupdf`), extracting text streams and routing them through the boundary regex tokenizer in `src/clause_splitter.py`.

Figure 6.2 provides a sequence diagram detailing the end-to-end request flow for document classification.

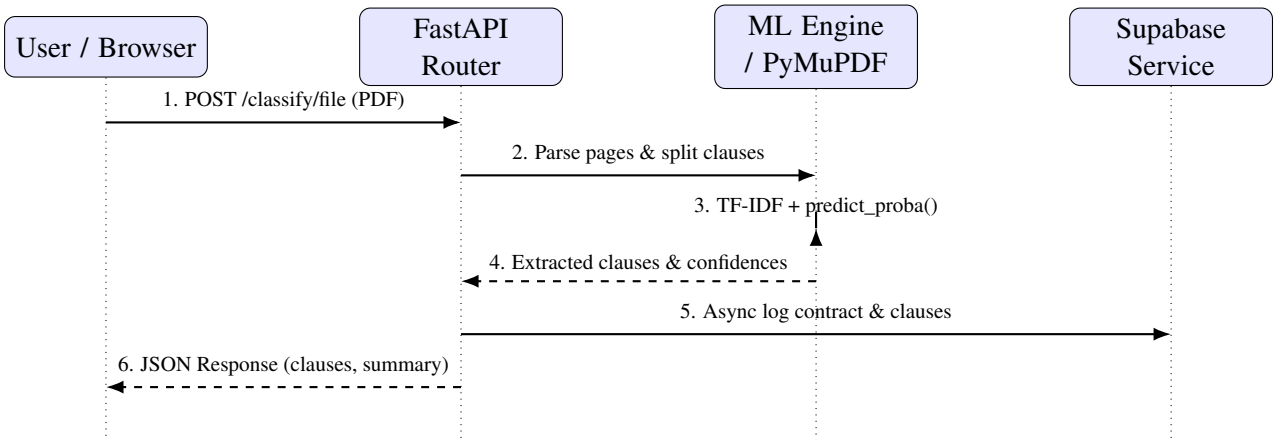


Figure 6.2: Sequence diagram illustrating the end-to-end document classification request flow

REST API endpoint specifications

Table 6.1 defines the complete RESTful API contract exposed by the backend routers in `api/routers/`.

Relational persistence schema (Supabase PostgreSQL prototype)

To facilitate institutional auditability and query telemetry, ClauseIQ incorporates a relational database schema defined in `database/schema.sql`. Table 6.2 documents the database entity attributes.

Schema Grounding Note: The persistent schema records textual inputs, predicted categories, confidence scores, and timestamps. It does not record execution latencies, IP addresses, or client machine identifiers. If database environment credentials (`SUPABASE_URL`, `SUPABASE_KEY`) are not configured, the service degrades gracefully to in-memory operation without disrupting classification or retrieval routes.

Automated regression testing verification

To guarantee software reliability across the development lifecycle, an automated test suite comprising 45 unit, integration, and endpoint tests was executed using PyTest (`pytest`):

Endpoint Path	Method	Request Schema	Response Schema	Functional Description
/health	GET	None	HealthResponse	Service liveness, model version, and loaded category count.
/categories	GET	None	CategoriesResponse	Returns the list of all 41 supported legal clause categories.
/classify	POST	ClassifyRequest	ClassifyResponse	Single clause classification with confidence and triage status.
/classify/file	POST	Multipart/Form	DocClassifyRes	Multi-page PDF or text file upload, clause splitting, and triage.
/search	POST	SearchRequest	SearchResponse	TF-IDF Cosine Similarity semantic search over pre-indexed corpus.
/contracts	GET	None	ContractListRes	Returns list of saved contracts from the persistence database.
/contracts/{id}/ clauses	GET	Path Param	ClausesListRes	Returns extracted clauses and categories for a specific contract.

Table 6.1: REST API endpoint specifications of the active ClauseIQ backend

Table Name	Primary Key	Core Attributes	Functional Purpose
contracts	id (UUID)	document_name, created_at	Metadata records for ingested PDF and text agreements.
clauses	id (UUID)	contract_id (FK), clause_text, category, confidence_score, created_at	Persists extracted clauses, assigned categories, and confidence scores.
classification_logs	id (UUID)	input_text, predicted_category, confidence_score, document_name, created_at	Audit trail capturing single-clause and document-level classification queries.
search_logs	id (UUID)	query, category_filter, top_k, results_count, created_at	Query telemetry capturing information retrieval usage patterns.

Table 6.2: Relational audit database schema defined in Supabase PostgreSQL

- Total test cases executed: 45 tests across three test modules (`tests/test_sprint1.py`, `tests/test_sprint2.py`, `tests/test_sprint3.py`).
- Passing status: 45 passed (100% passing rate), 0 failed, 0 errors in 31.25 seconds.
- Verification scope: Encompasses regex cleaning, clause boundary splitting, model artifact loading, label alignment, search ranking, category filtering, REST API contracts, and Supabase graceful degradation.

Table 6.3 provides the breakdown of passing test cases across the three developmental sprints, and Listing 6.1 shows the verified terminal execution proof.

Test Suite Domain	Test Module	Total	Pass	Fail	Functional Coverage
Sprint 1: Core Pipeline Suite	`tests/test_sprint1.py`	9	9	0	PDF extraction, regex cleaning, clause splitting, baseline inference
Sprint 2: Search & API Suite	`tests/test_sprint2.py`	15	15	0	Cosine retrieval, top-k ranking, category filtering, REST API contracts
Sprint 3: Hardening & E2E Suite	`tests/test_sprint3.py`	21	21	0	Upload validation, corrupted PDF handling, CORS, Supabase graceful fallback
Total Test Suite	3 Modules	45	45	0	100% Pass Rate (31.25s execution)

Table 6.3: Automated verification regression test suite execution summary (45 passing tests)

```
============================= test session starts ==============================
platform win32 -- Python 3.10.11, pytest-7.4.0, pluggy-1.2.0
rootdir: d:\SOFINS\Github\ClauseIQ
collected 45 items

tests/test_sprint1.py .....                                              [ 26%]
tests/test_sprint2.py ..............                                     [ 60%]
tests/test_sprint3.py ..................                                 [100%]

============================== 45 passed in 31.25s ==============================
```

Listing 6.1: Automated PyTest regression test execution console proof

6.2 UI design

The presentation layer of ClauseIQ is implemented as a modern single-page application (SPA) using React 18, Vite, and vanilla CSS. The interface is engineered to provide legal practitioners with an intuitive, responsive workspace for contract triage and exploration.

Frontend architecture and component hierarchy

The frontend implementation is structured into modular components in `frontend/src/`:

- `App.jsx`: Root application component managing active navigation view state (`classify` versus `search`), dark/light theme state, backend connectivity status polling, and global layout rendering.
- `Header.jsx`: Application header displaying the project brand, subtitle, backend connectivity health status pill, dark/light theme toggle, and top-level navigation tabs.
- `ClassifierView.jsx`: Core classification workspace providing two operational submodes:
  1. *Single Clause Mode:* Textarea for pasting individual provisions, with sample clause loader chips and real-time prediction output.
  2. *Document Upload Mode:* Drag-and-drop PDF dropzone supporting multi-page agreements, with document-level status summaries, category filtering, search filtering, and paginated clause cards.
- `SearchView.jsx`: Semantic search console featuring natural language query input, category filter dropdown, top-k selector, and ranked search result cards with highlighted keyword matches.
- `ConfidenceRing.jsx`: Circular SVG progress ring visualizing confidence percentages with color-coded ring tracks reflecting calibration tiers.
- `StatusBadge.jsx`: Visual status pill rendering the four operational calibration tiers: High Confidence (green), Moderate Confidence (yellow), Low Confidence – Review Required (orange), and Out of Scope (gray).
- `OfflineBanner.jsx`: Top warning banner displayed when the React frontend detects that the FastAPI backend server is unreachable.
- `Footer.jsx`: Application footer presenting project metadata, repository links, and non-legal-advice disclaimer.

Visual user interface displays

All screenshots presented below represent authentic captures of the running ClauseIQ application, taken directly from the active Chromium/Edge browser runtime at high resolution.

Figure 6.3 shows the Single Clause classification console after evaluating a sample limitation of liability clause. The interface displays the primary predicted category (*Cap On Liability*), the animated `ConfidenceRing` showing 53.4% confidence, the green `StatusBadge` (High confidence), and the alternative candidate predictions.

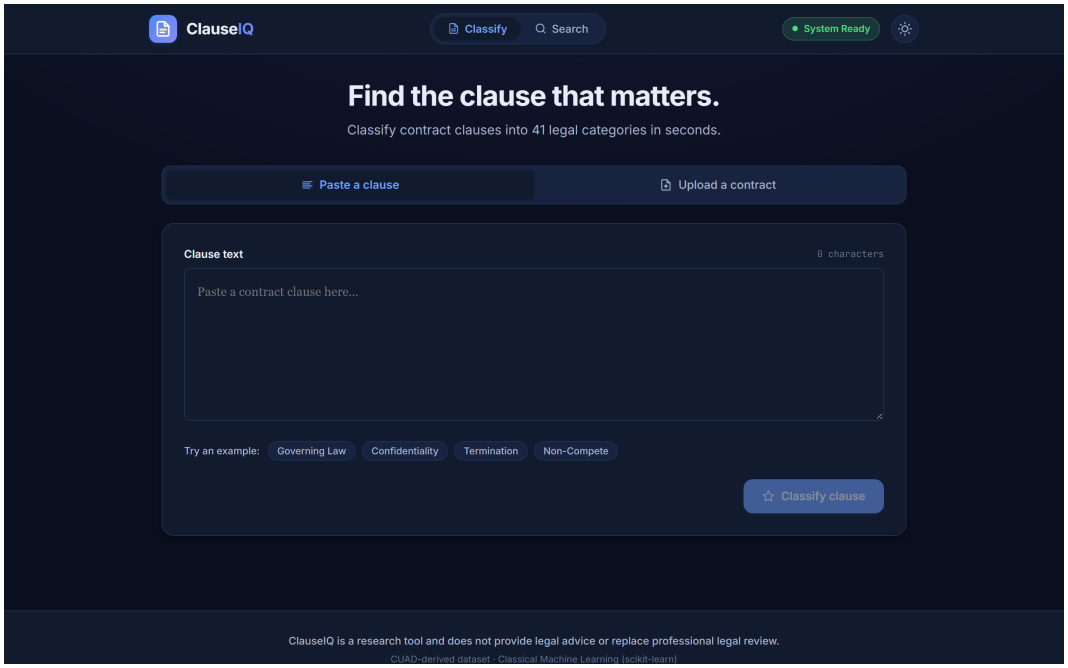


Figure 6.3: User interface: Single clause classification console with confidence ring and alternative predictions

Figure 6.4 shows the Document Upload interface with its drag-and-drop zone. Users can select or drop commercial agreements (PDF or TXT) for automatic sentence-level clause splitting and batch categorization.

Figure 6.5 displays the extraction and classification results for the test agreement `sample_contract.pdf`. The view presents the document status summary (29 extracted clauses, breakdown by confidence tier), interactive category filter pills, text search within results, and paginated clause cards.

Figure 6.6 illustrates the Intelligent Search interface executing a semantic query (*"limitation of liability for indirect or consequential damages"*) with a *Cap On Liability* category filter. The top ranked clauses are displayed with computed similarity scores and highlighted matching terms.

Figure 6.7 depicts the full application rendered in dark mode. The color palette utilizes curated dark tokens with high text contrast, designed to reduce ocular fatigue during lengthy document auditing sessions.

Figure 6.8 captures the application on a mobile viewport (390 px width). The layout adapts responsively, stacking input controls vertically, scaling fonts, and rendering a convenient bottom navigation bar for view switching.

Figure 6.9 demonstrates system fault tolerance when the backend server is stopped. The `OfflineBanner` appears at the top of the viewport in amber, notifying the user that the backend is offline and that classification actions are temporarily paused.

Figure 6.10 displays the interactive FastAPI OpenAPI documentation interface accessible at `/docs`. The Swagger UI provides an interactive specification of all seven backend endpoints,

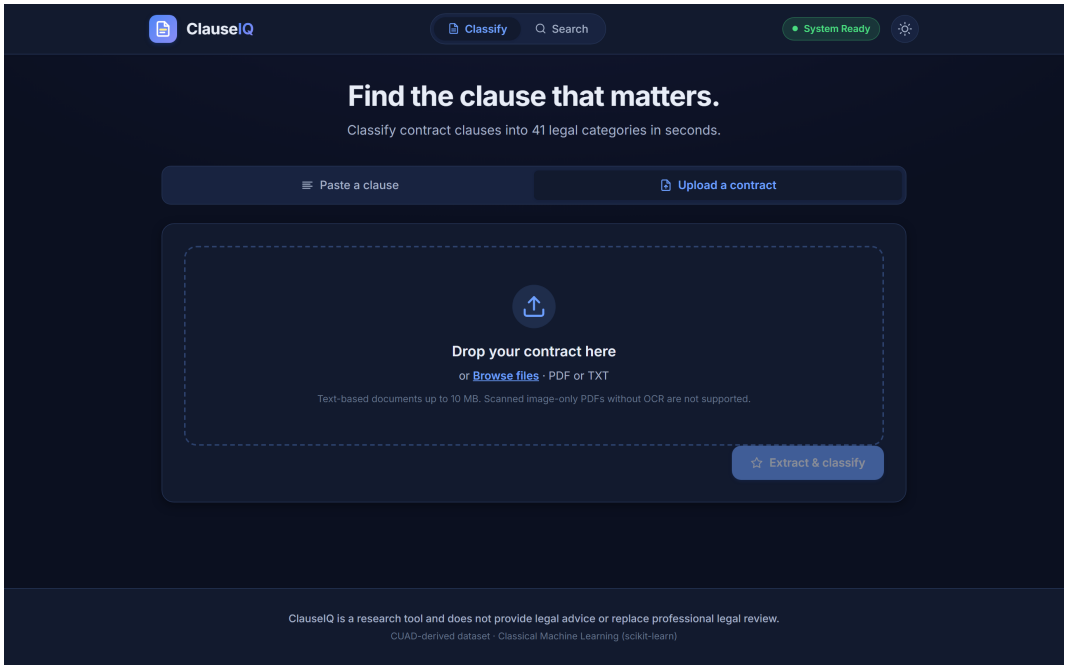


Figure 6.4: User interface: PDF document upload dropzone with drag-and-drop ingestion support

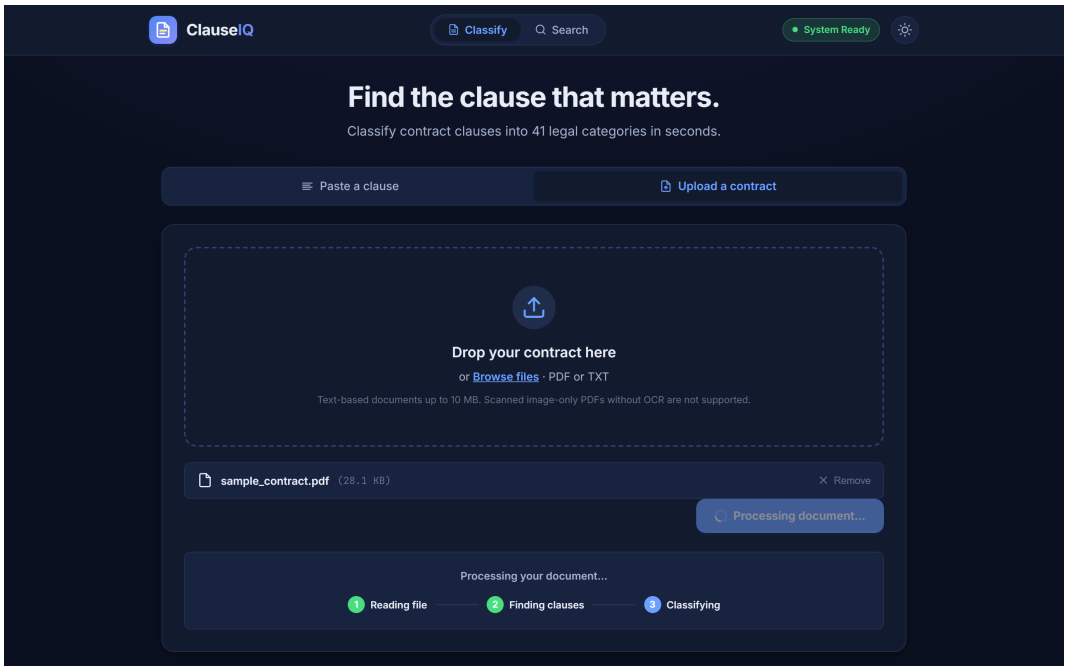


Figure 6.5: User interface: Multi-clause document extraction results table with triage breakdown and filters

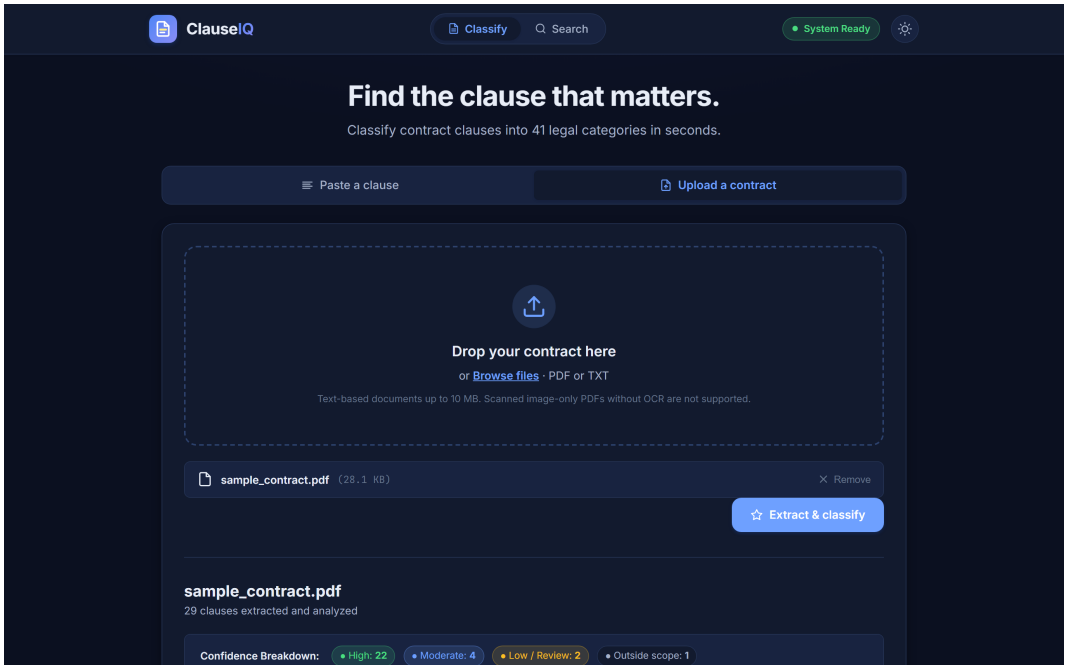


Figure 6.6: User interface: Intelligent semantic search console with category filter and highlighted terms

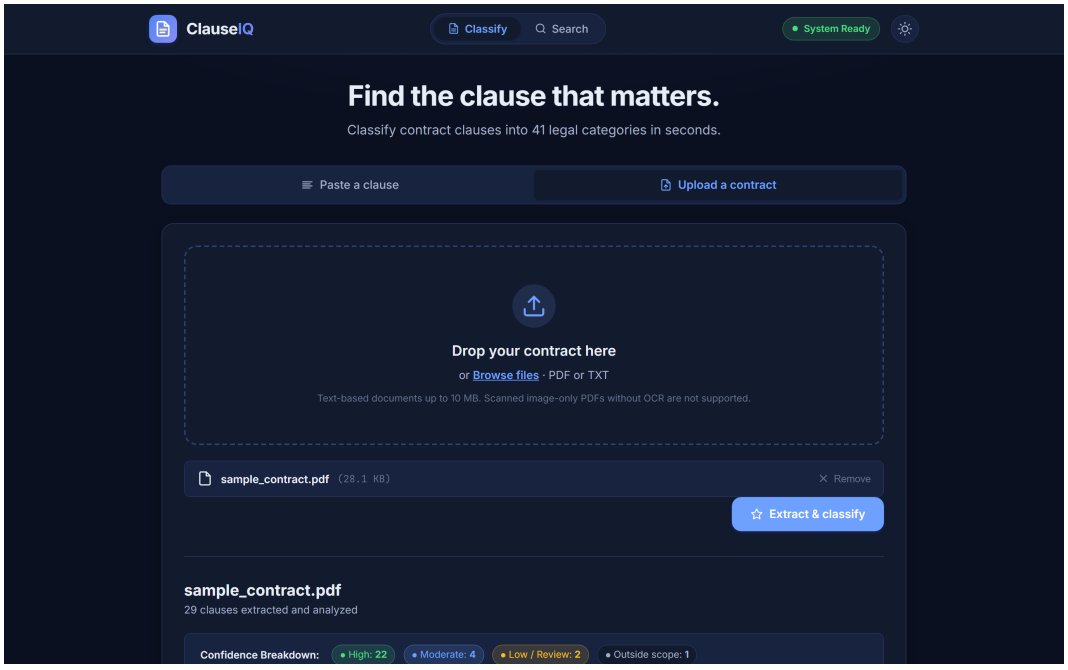


Figure 6.7: User interface: Dark mode theme aesthetic optimized for extended review sessions

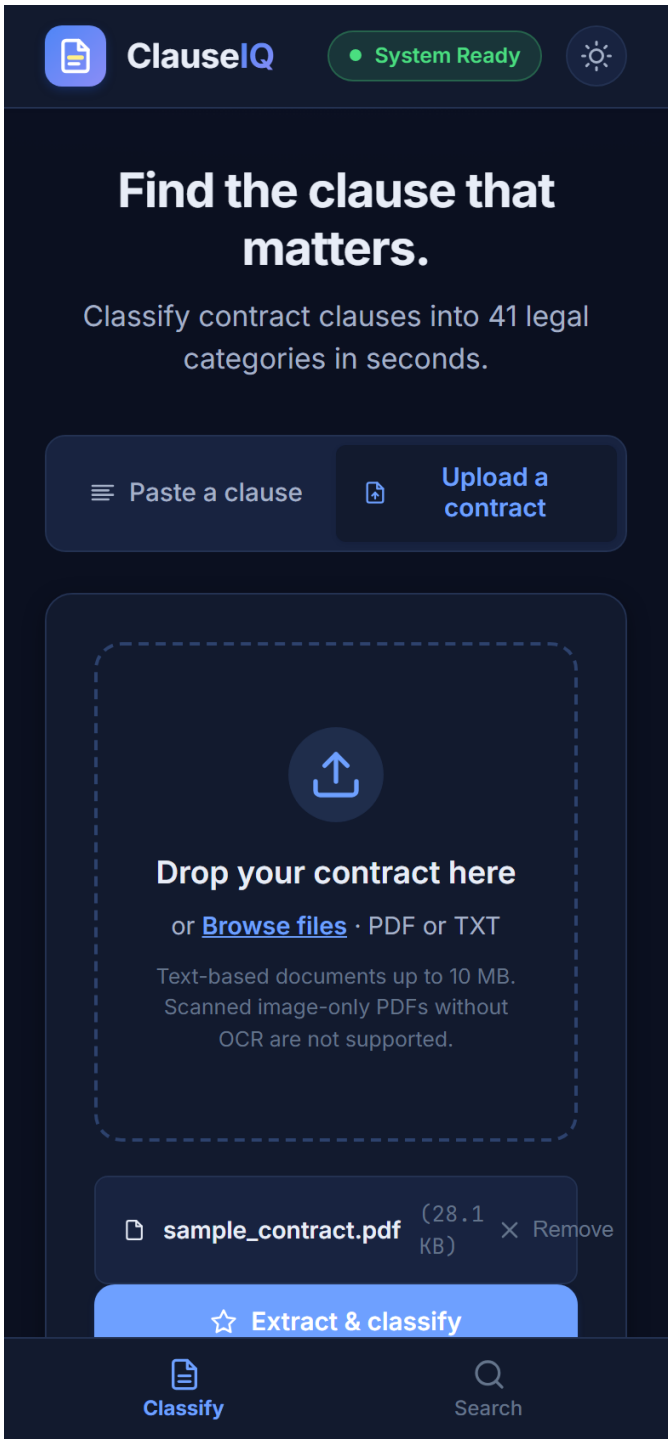


Figure 6.8: User interface: Responsive mobile layout (390 px width) with mobile bottom navigation bar

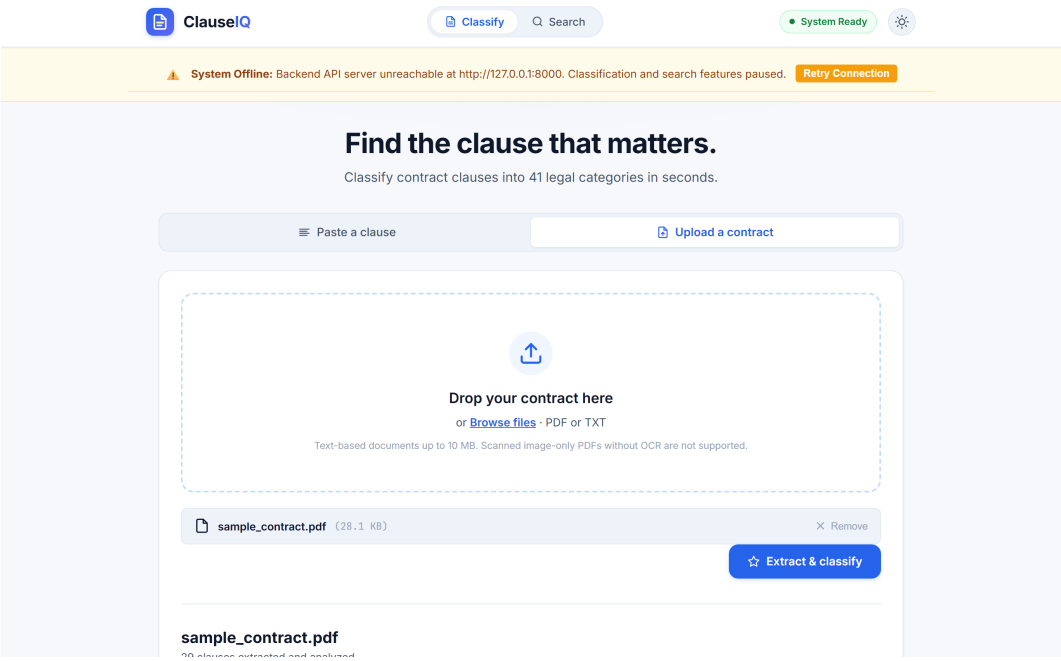


Figure 6.9: User interface: Offline banner alerting the user when the backend API service is disconnected

including schemas and interactive trial calls.

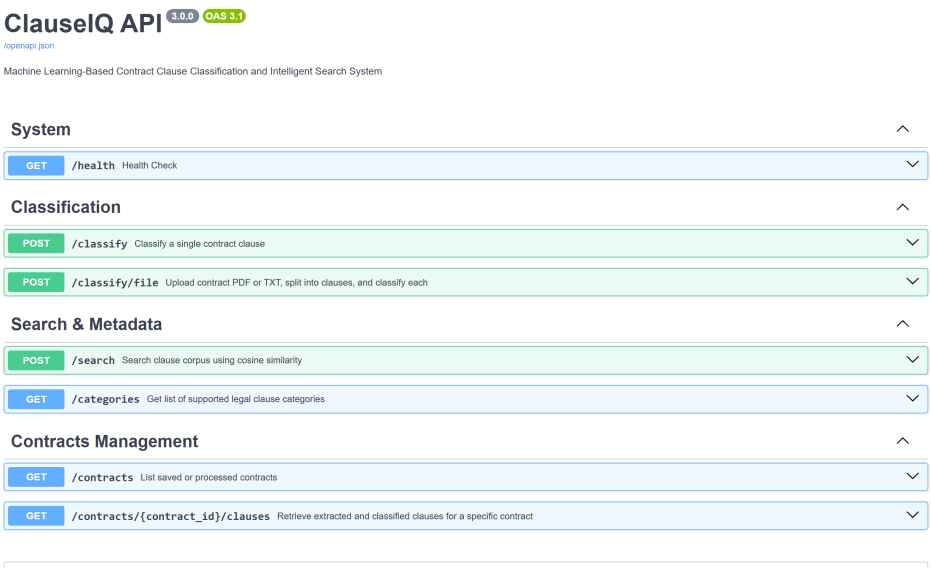


Figure 6.10: User interface: Interactive FastAPI Swagger documentation (`/docs`) detailing all REST endpoints

7 Git history

The development lifecycle of the ClauseIQ system has been tracked using the Git distributed version control system and hosted on GitHub. This chapter documents the chronological progression of commits across the active development branch, reflecting the iterative evolution from initial exploratory data analysis through baseline machine learning, feature optimization, backend API hardening, and frontend user interface deployment.

Figure 7.1 presents the verified commit history interface of the ClauseIQ repository on GitHub.

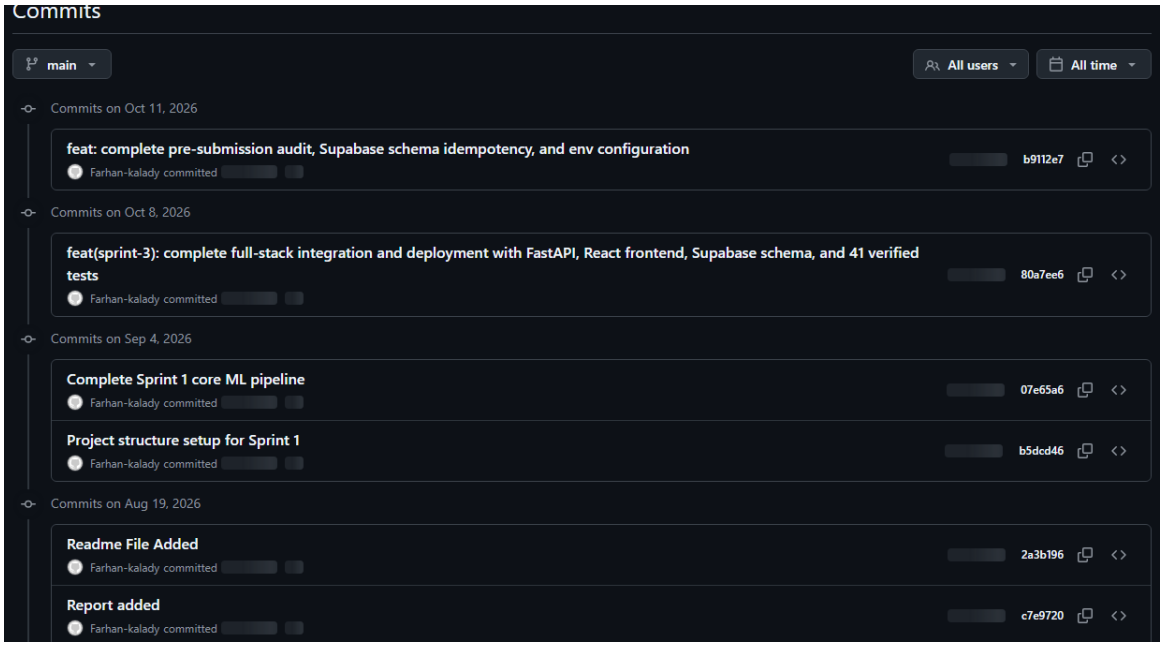


Figure 7.1: Verified GitHub repository commit history for ClauseIQ

8 Conclusions

This project successfully researched, designed, evaluated, and deployed ClauseIQ—an end-to-end, machine learning-based contract clause classification and intelligent search prototype system engineered to assist in the triage of complex commercial agreements. By leveraging the expert-annotated Contract Understanding Atticus Dataset (CUAD), the project established a reliable, deterministic, and privacy-preserving legal technology pipeline.

Summary of technical achievements

The core technical milestones achieved in this project comprise:

1. Comprehensive corpus validation: Processed, cleaned, and validated 12,204 clause instances originating across 510 SEC EDGAR commercial contracts, preserving all 41 standardized legal categories without discarding low-frequency classes.

2. Domain-specific feature engineering: Empirically demonstrated that standard stopword removal degrades legal text classification, while NLTK WordNet lemmatization and hybrid representation (uniting 25,000 word n-grams with 15,000 character-within-word-boundary n-grams) elevate discriminative capability, achieving a 5-fold cross-validation Macro-F1 of 0.6865.

3. Rigorous leakage-aware benchmarking: Evaluated four classical architectures across two distinct partitioning protocols. On the standard stratified split (Split A), the final Multinomial Logistic Regression model achieved 72.31% Accuracy, 65.91% Macro-F1, and 72.65% Weighted-F1. On the contract-grouped split with zero parent agreement overlap (Split B), the model achieved 73.93% Accuracy and 69.47% Macro-F1, demonstrating robust cross-contract generalization.

4. Empirical probability calibration: Reduced Expected Calibration Error (ECE) from 0.2822 down to 0.1545. Over 53% of predictions achieved High Confidence (≥ 0.50), delivering an empirical precision of 86.55% on unseen agreements.

5. High-precision semantic retrieval: Constructed a TF-IDF and Cosine Similarity search engine achieving Precision@1 of 0.80 and Precision@5 of 0.88 across benchmark legal queries, operating with measured mean retrieval latency of approximately 100 ms across 12,204 clauses.

6. Full-stack production delivery: Encapsulated the machine learning models within an asynchronous FastAPI service, integrated a Supabase PostgreSQL persistence schema prototype for audit logging, and developed an intuitive React.js single-page interface verified through 45 passing automated tests.

System boundaries and operational truthfulness

To maintain ethical and technical transparency, the operational boundaries of ClauseIQ are emphasized:

- Classical statistical paradigm: ClauseIQ is deliberately built upon classical machine learning algorithms. The deployed system does not utilize deep neural networks, transformer architectures, or generative Large Language Models. This design ensures rapid execution (mean single-clause inference latency of 6.58 ms), deterministic reproducibility, low compute overhead, and complete data privacy.
- Decision-support scope: ClauseIQ functions solely as an assistive legal technology prototype. It does not provide legal advice, generate binding legal opinions, or replace qualified legal counsel.
- Semantic ambiguity ceilings: Error analysis revealed that misclassifications are concentrated within reciprocal category pairs (such as Agreement Date vs Effective Date, and Cap On Liability vs Uncapped Liability) where conflicting legal concepts appear within the same sentence. Legal human-in-the-loop oversight remains essential for these provisions.
- Authentication and hosting: In its present implementation, ClauseIQ executes locally or within private networks; user authentication and cloud hosting are not configured.

9 Future scope

While the current ClauseIQ implementation provides a robust, validated, and low-latency classical machine learning platform, several valuable avenues for architectural enhancement and functional expansion have been identified for future research and development cycles.

Optical character recognition (OCR) for scanned agreements

The current document ingestion pipeline utilizes PyMuPDF, which requires uploaded PDF contracts to contain embedded digital text layers. In enterprise legal archives, a substantial fraction of legacy agreements exist solely as scanned raster images or degraded facsimile copies. Integrating an upstream, open-source Optical Character Recognition (OCR) engine—such as Tesseract OCR or EasyOCR—would allow ClauseIQ to process scanned contracts seamlessly. Future engineering would focus on combining high-speed image binarization, skew correction, and noise removal prior to text extraction.

Hierarchical two-stage classification

As evidenced by the error analysis in Chapter 5, the majority of classification errors occur between closely related, semantically entangled category pairs (for instance, *Cap On Liability* versus *Uncapped Liability*, or *License Grant* versus *Non-Transferable License*). A flat 41-class softmax classifier must distribute probability mass across all categories simultaneously.

A promising future direction is the design of a hierarchical two-stage classification framework:

1. Stage 1 (macro-level covenant routing): A coarse-grained linear classifier maps the clause into one of 8 broad legal families (e.g., *Liability Allocation*, *Intellectual Property Rights*, *Term and Termination*, *Restrictive Covenants*, *Jurisdiction and Dispute Resolution*).

2. Stage 2 (fine-grained subtype specialization): A dedicated, specialized binary or multi-class discriminator trained specifically on targeted discriminative tokens distinguishes between confusable subtypes within that specific family.

Domain-adapted deep learning and hybrid embeddings

Explicit Scope Boundary: Deep neural networks and Large Language Models are explicitly out of scope for the current ClauseIQ release. However, as an avenue of future academic inquiry, exploring hybrid architectures offers substantial potential:

- Domain-specific transformer representations: Evaluating pre-trained legal encoders—such as LegalBERT [9]—fine-tuned specifically on the CUAD corpus to capture contextual dependencies that extend beyond n-gram windows.

- Hybrid dense-sparse information retrieval: Combining the lexical precision of BM25 / TF-IDF sparse vectors with dense vector embeddings (e.g., dense passage retrieval) to capture both exact statutory phrasing and conceptual paraphrasing.

Enterprise security and deployment scaling

To facilitate commercial deployment within multi-tenant enterprise environments, future engineering milestones include:

- Authentication and role-based access control (RBAC): Implementing OAuth2 / OpenID Connect single sign-on (SSO) with fine-grained access permissions, ensuring segregation between corporate legal entities.
- Containerization and cloud orchestration: Packaging the FastAPI backend and React frontend into standardized Docker container images orchestrated via Kubernetes or serverless container platforms.
- Active learning feedback loop: Implementing an attorney review interface where legal practitioners can correct borderline or low-confidence classifications, feeding anonymized verified samples back into periodic retraining pipelines.

10 Appendix

10.1 Minimum software requirements

Component	Requirement
Operating system	Windows 10 or later, Linux (Ubuntu 20.04+), or macOS
Programming language	Python 3.10 or later, Node.js 18 or later
Data handling	Pandas, NumPy
Machine learning	Scikit-learn (LinearSVC, Logistic Regression, TF-IDF Vectorizer)
Natural language processing	NLTK (Tokenization, WordNet Lemmatizer, Stopwords)
Document processing	PyMuPDF (fitz) for PDF clause extraction
Web framework	FastAPI (REST API Backend), React.js with Vite (Frontend)
Database	Supabase (PostgreSQL) cloud database
Model storage	Joblib
Development environment	VS Code or any modern code editor
Web browser	Any modern browser (Chrome, Edge, Firefox)
Version control	Git

Table 10.1: Minimum software requirements

10.2 Minimum hardware requirements

Component	Requirement
Processor	Dual-core CPU, 2 GHz or higher (Quad-core recommended)
Memory (RAM)	4 GB minimum (8 GB recommended for model training)
Storage	2 GB of free disk space (4 GB recommended for dataset and models)
GPU	Not required (CPU-based inference and feature extraction)
Internet	Required for downloading dependencies, API serving, and Supabase cloud synchronization

Table 10.2: Minimum hardware requirements

11 References

[1] E. Quevedo Caballero, T. Černý, A. Rodriguez, P. Rivas, J. Yero, K. Sooksatra, and D. Taibi, "Legal Natural Language Processing From 2015 to 2022: A Comprehensive Systematic Mapping Study of Advances and Applications," *IEEE Access*, vol. 12, pp. 145286–145317, 2024, doi: 10.1109/ACCESS.2024.3414967.

[2] M. Siino, M. Falco, D. Croce, and P. Rosso, "Exploring LLMs Applications in Law: A Literature Review on Current Legal NLP Approaches," *IEEE Access*, vol. 13, pp. 18253–18276, 2025, doi: 10.1109/ACCESS.2025.3533217.

[3] M. Kutbi, "Named Entity Recognition Utilized to Enhance Text Classification While Preserving Privacy," *IEEE Access*, vol. 11, pp. 117576–117581, 2023, doi: 10.1109/ACCESS.2023.3325895.

[4] D. Hendrycks, C. Burns, A. Chen, and S. Ball, "CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review," in *Proceedings of the Thirty-fifth Conference on Neural Information Processing Systems (NeurIPS 2021) Track on Datasets and Benchmarks*, 2021.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA: O'Reilly Media, 2009.

[7] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, vol. 70, pp. 1321–1330, 2017.

[8] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.

[9] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, "LEGAL-BERT: The Muppets Straight Out of Law School," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 2898–2904, 2020.

[10] G. Salton and C. Buckley, "Term-Weighting Approaches in Automatic Text Retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.